

Woxsen University

School of Technology



B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Tailoring and Boutique Order Management System (Lumière Boutique)

Course Code: 25TU03MJM0

Course Name: Database Management Systems

Semester: 3

Academic Year: 2025 – 29

Submitted by:

Name: Aarya Dubey , D. Neshitha

Register Number: 25WU0101003, 25WU0101028

Section: CSE Tigers

TABLE OF CONTENTS

1. Abstract	2
2. Introduction and Problem Statement	2
2.1 Introduction	2
2.2 Problem Statement	3
3. Objectives and Scope	3
3.1 Objectives	3
3.2 Scope of the Project	4
3.3 Scope Limitations	5
4. Software & Hardware Requirements	6
4.1 Software Requirements	6
4.2 System Requirements for Hardware	6
4.3 Development Environment	6
5. ER Diagram	7
5.1 Major Entities and Relationships	7
5.2 ER Diagram Representation	8
5.3 Important Relationships	8
6. Relational Schema and Normalization	9
6.1 Relational Schema	9
6.2 Primary Keys and Foreign Keys	10
6.3 Normalization	11
7. Data Dictionary	14
7.1 CUSTOMERS	14
7.2 MEASUREMENT_PROFILES	15
7.3 GARMENT_TYPES	15
7.4 DESIGNS	15
7.5 FABRICS	16
7.6 FABRIC_STOCK	16
7.7 ORDERS	16
7.8 ORDER_ITEMS	17
7.9 FABRIC_ISSUES	17
7.10 TAILORS	17
7.11 TAILOR_ASSIGNMENTS	18
7.12 TRIALS	18
7.13 ALTERATIONS	18
7.14 INVOICES	19
7.15 PAYMENTS	19
8. SQL Commands Used	20

8.1 DDL Commands	20
8.2 DML Commands	21
8.3 Sample Output	22
9. Queries with Outputs	23
9.1 Query 1 – Customer Order History	23
9.2 Query 2 – Orders with Garment Types and Designs	24
9.3 Query 3 – Orders Due for Delivery Within 7 Days	24
9.4 Query 4 – Tailor Workload and Assignment Breakdown	25
9.5 Query 5 – Fabric Reorder Threshold Alert	25
9.6 Query 6 – Order Status Distribution Across Workflow	26
9.7 Query 7 – Total Alteration Expenditure Analysis	26
9.8 Query 8 – Maximum Order Value (Nested Subquery)	27
9.9 Query 9 – Orders with Tailor and Fitting Stage Details	27
9.10 Query 10 – Invoices Payment Collection and Outstanding Balance ..	28
9.11 Query 11 – Fabric Utilization per Garment Order	28
9.12 Query 12 – View-based Business Intelligence Reporting	29
10. UI Design and Screenshots	30
10.1 Application Launch & Aesthetic Design	30
10.2 Boutique Overview Dashboard	30
10.3 Customer Management Screen	31
10.4 Measurement Profiles Screen	31
10.5 Fabrics & Stock Inventory Screen	32
10.6 Order Booking & Workflow Screen	32
10.7 Fitting Trials & Alterations Screen	33
10.8 Billing, Invoicing & Payments Screen	33
10.9 Reports & Analytics Screen	34
10.10 Database Reflection & Synchronization Test	34
11. Implementation Details	35
11.1 Technology Stack	35
11.2 Application Architecture	35
11.3 Database Connection & Management	36
11.4 Transactions, DAO and Business Logic	36
11.5 Business Rule Triggers and Integrity Constraints	37
12. Testing	38
12.1 Functional Testing	38
12.2 Database Testing	39
12.3 Transaction and Trigger Testing	39
12.4 UI Testing	40
12.5 Test Results	40

13. Conclusion and Future Enhancements	41
13.1 Conclusion	41
13.2 Future Enhancements	41
14. References	42
15. Contribution of Each Member	43
16. GitHub Repository	43

LIST OF TABLES

Table 4.1: Software and Technology Requirements	6
Table 5.1: Important Relationships between Entities	9
Table 6.1: Primary Keys and Foreign Keys	10
Table 6.2: Unnormalized Form (UNF) – Sample Order Data	11
Table 6.3: First Normal Form (1NF) – Sample Order Data	12
Table 6.4: Second Normal Form (2NF) – ORDERS	12
Table 6.5: Second Normal Form (2NF) – ORDER_ITEMS	13
Table 6.6: Third Normal Form (3NF) – ORDERS	13
Table 6.7: Third Normal Form (3NF) – CUSTOMERS	13
Table 6.8: Third Normal Form (3NF) – ORDER_ITEMS	14
Table 6.9: Third Normal Form (3NF) – FABRICS & DESIGNS	14
Table 6.10: Third Normal Form (3NF) – TAILORS	14
Table 7.1: CUSTOMERS Table Data Dictionary	14
Table 7.2: MEASUREMENT_PROFILES Table Data Dictionary	15
Table 7.3: GARMENT_TYPES Table Data Dictionary	15
Table 7.4: DESIGNS Table Data Dictionary	15
Table 7.5: FABRICS Table Data Dictionary	16
Table 7.6: FABRIC_STOCK Table Data Dictionary	16
Table 7.7: ORDERS Table Data Dictionary	16
Table 7.8: ORDER_ITEMS Table Data Dictionary	17
Table 7.9: FABRIC_ISSUES Table Data Dictionary	17
Table 7.10: TAILORS Table Data Dictionary	17
Table 7.11: TAILOR_ASSIGNMENTS Table Data Dictionary	18
Table 7.12: TRIALS Table Data Dictionary	18
Table 7.13: ALTERATIONS Table Data Dictionary	18
Table 7.14: INVOICES Table Data Dictionary	19
Table 7.15: PAYMENTS Table Data Dictionary	19
Table 11.1: Technology Stack	35
Table 12.1: Functional Testing – Test Cases	38
Table 12.2: Test Results	40
Table 15.1: Contribution of Each Member	43

LIST OF FIGURES

Figure 5.1: Tailoring Boutique Management System ER Diagram	8
Figure 8.1: Database and Table Creation	22
Figure 8.2: DML Operation and Output	22

Figure 9.1: Query 1 Execution – Customer Order History	23
Figure 9.3: Query 3 Execution – Orders Due for Delivery Within 7 Days ...	24
Figure 9.4: Query 4 Execution – Tailor Workload and Assignment Breakdown ..	25
Figure 9.8: Query 8 Execution – Maximum Order Value (Nested Subquery) ...	27
Figure 9.10: Query 10 Execution – Invoices Payment Collection and Balance ..	28
Figure 9.12: Query 12 Execution – View-based Business Intelligence Reporting ..	29
Figure 10.1: Boutique Performance Dashboard Screen	30
Figure 10.2: Customer Management Directory Screen	31
Figure 10.3: Bespoke Measurement Profiles Screen	31
Figure 10.4: Fabrics Catalogue and Stock Inventory Screen	32
Figure 10.5: Order Booking and Production Workflow Screen	32
Figure 10.6: Fitting Trials and Alteration Scheduling Screen	33
Figure 10.7: Invoicing, Billing and Payment Screen	33
Figure 10.8: Strategic Database View Reports Screen	34
Figure 11.1: Python MySQL Database Connection Layer	36
Figure 11.2: DAO Operations and Transaction Execution with Rollback	36
Figure 11.3: Database Business Rule Triggers (Fabric Stock & Payment Caps) ..	37
Figure 12.1: Duplicate Customer Code Constraint Validation Test	39
Figure 12.2: Delivery Date Check Constraint Validation Test	39
Figure 12.3: Fabric Issue Stock Exceeded Trigger Test	40
Figure 12.4: Payment Overcharge Exceeding Invoice Balance Trigger Test ..	40

1. ABSTRACT

The Tailoring and Boutique Order Management System (commercially branded as Lumière Boutique) is an advanced desktop database software solution engineered to digitize, unify, and automate the entire operational lifecycle of modern custom fashion boutiques and artisanal tailoring studios. The primary objective of the software is to eliminate the severe vulnerabilities of manual bookkeeping, paper measurement index cards, and disjointed spreadsheets by centralizing customer records, anatomical measurement profiles, garment catalogs, designer styles, fabric roll inventory, order fulfillment pipelines, tailor task assignments, client fitting trials, alteration tracking, automated billing, and ledger payment collections.

The software architecture establishes an enterprise-grade role-based desktop environment pairing a highly responsive graphical user interface developed in Python (using Tkinter and ttk with a curated blush-pink and clean-white boutique visual theme) with a robust MySQL 8.0 relational database engine. Connectivity between the interface and the database is governed through a resilient Data Access Object (DAO) architecture leveraging `mysql.connector`, ensuring parameterized queries, automated transaction handling with `COMMIT` and `ROLLBACK` safeguards, and centralized exception dispatching.

The relational database schema is rigorously modeled in accordance with the principles of Third Normal Form (3NF), encompassing fifteen normalized tables. Referential integrity, domain constraints, and automated business policies are strictly enforced through primary keys, composite keys, foreign keys with `ON DELETE CASCADE` and `SET NULL` rules, `CHECK` constraints for positive dimensions and financial values, and `UNIQUE` constraints preventing duplicate customer records, overlapping tailor trial appointments, and conflicting line items. Furthermore, event-driven ACID database triggers are integrated into the MySQL engine to automatically restrict fabric issues from exceeding available roll stock and prevent overpayments beyond outstanding invoice balances.

In conclusion, the system delivers complete end-to-end transparency, eliminates data redundancy, guarantees inventory and billing consistency, and empowers boutique managers with actionable operational insights through pre-compiled dynamic database views and statistical aggregations.

2. Introduction and Problem Statement

2.1 Introduction

The bespoke tailoring and custom boutique industry is fundamentally distinct from standard off-the-rack garment retail. In custom tailoring, each garment is individually produced to fit the exact anatomical dimensions, aesthetic choices, and fabric preferences of the individual client. Consequently, managing a tailoring establishment entails coordinating massive volumes of heterogeneous, highly interdependent data across multiple operational departments.

A standard customer journey begins with recording detailed anatomical measurements (bust, waist, hip, shoulder width, sleeve length, and total garment length). The client selects a garment type (e.g., Kurti, Blouse, Lehenga, Salwar Suit, Evening Gown), chooses an embroidery or cut design pattern, and selects specific fabric rolls from the shop's inventory. Once the order is logged, available yardage must be deducted, a master tailor with suitable specialization must be allocated, fabric issues must be tracked, and fitting trials must be scheduled. Following the client trial session, adjustments and alterations are recorded with supplementary costs, culminating in final invoice generation and payment settlement.

The Tailoring and Boutique Order Management System provides a unified, centralized database application specifically crafted for boutique administrators, master cut-tailors, and cashiers to manage this intricate operational pipeline effortlessly.

2.2 Problem Statement

In traditional boutiques, day-to-day operations rely on physical notebooks, handwritten paper measurement slips, and basic spreadsheets. This manual paradigm creates critical inefficiencies, financial leakages, and service delays:

- **Loss of Customer Measurement Cards:** Paper measurement slips are frequently misplaced, damaged, or updated inconsistently, resulting in ill-fitting garments and costly fabric waste.
- **Lack of Real-Time Fabric Inventory Tracking:** Shopkeepers cannot accurately determine whether sufficient yardage of a specific fabric roll is physically available before accepting a customer order, leading to broken delivery promises.
- **Tailor Scheduling Conflicts & Trial Double-Booking:** Fitting appointments frequently overlap because receptionists have no visibility into individual tailor schedules, leading to tailor double-booking and disgruntled clients waiting in boutique salons.
- **Opacity in Order Production Workflow:** Managers cannot monitor which orders are PLACED, currently IN_PROGRESS, awaiting TRIAL, or stalled in ALTERATION, leading to missed client delivery deadlines.
- **Alteration Cost Leakages & Billing Discrepancies:** Post-trial alterations incur labor and material costs that are frequently forgotten or miscalculated during final checkout, causing direct revenue losses.
- **Inaccurate Outstanding Balance & Payment Ledgering:** Accepting partial advance deposits without automated ledger verification allows customers to collect garments while balances remain unpaid.
- **Absence of Business Intelligence:** Compiling weekly or monthly performance reports requires days of manual tallying, leaving boutique proprietors blind to revenue trends and tailor productivity.

Therefore, there exists a critical necessity for a modernized, centralized Relational Database Management System (RDBMS) paired with an intuitive graphical interface to enforce ACID transactional consistency, automate stock calculations, enforce business constraints, and provide comprehensive reporting.

3. Objectives and Scope

3.1 Objectives

The key objectives of the Tailoring and Boutique Order Management System are as follows:

- **1. Centralization of Client Data:** Store all customer profiles, contact credentials, and anatomical measurements in unified relational entities with referential integrity.
- **2. Catalog and Inventory Management:** Maintain a structured catalog of garment silhouettes, neckline/cut designs, and available fabric rolls.
- **3. Fabric Inventory Automation:** Track meters in stock and enforce automatic stock depletion upon fabric issuance, preventing fabric over-issuance.
- **4. Order Pipeline Orchestration:** Record multi-item customer bookings with itemized pricing, fabric requirements, and committed delivery dates.
- **5. Tailor Workload Allocation:** Match garments to specialized artisans and track pending versus completed workloads per tailor.
- **6. Conflict-Free Trial Scheduling:** Guarantee through database-level UNIQUE constraints that a tailor cannot be booked for two fitting trials at the identical date and time slot.
- **7. Alteration Tracking & Financial Rollup:** Log alteration instructions and associated technician charges, automatically rolling costs into invoices.
- **8. Ledger & Payment Integrity:** Enforce that cumulative payment installments cannot exceed the final invoice balance through MySQL triggers.
- **9. Normalization to 3NF:** Design the relational schema according to Third Normal Form (3NF) to eliminate update, insertion, and deletion anomalies.
- **10. Strategic Business Reporting:** Implement pre-compiled database views for delivery-due orders, pending pipelines, tailor workloads, and monthly revenue.

3.2 Scope of the Project

The functional scope encompasses all core operations conducted within a boutique:

- **A. Customer & Measurement Management:** Captures customer identification, unique contact numbers, emails, addresses, and full anatomical measurement profiles.
- **B. Design Catalog & Fabric Inventory:** Maintains garment categories, design pattern catalogs, fabric rolls, unit prices per meter, and stock reorder thresholds.

- **C. Order Processing & Fabric Issuance:** Handles multi-item order creation, line-level garment pricing, fabric issue transactions, and delivery date checks.
- **D. Tailor Workload Management:** Manages tailor master profiles, specializations, status flags, and individual order assignments.
- **E. Trials & Alterations Pipeline:** Schedules fitting trials, updates trial outcomes, and logs itemized alterations with associated labor costs.
- **F. Billing, Invoicing & Payments:** Generates finalized invoices summing base items and alterations, recording multi-mode payments (CASH, UPI, CARD, BANK_TRANSFER).
- **G. Management Views & Reports:** Generates actionable analytical views covering pending orders, delivery alerts, tailor capacity, and financial audits.

3.3 Scope Limitations

The current implementation is architected as an on-premises desktop client application utilizing Python Tkinter connected via native TCP sockets to a MySQL relational database server. Distributed web frameworks (e.g., React, Node.js, Spring Boot) and external cloud microservices are excluded from the current release to prioritize zero-latency local operations, high data privacy, and direct SQL transactional control on boutique shop-floor workstations.

4. Software & Hardware Requirements

4.1 Software Requirements

The software and environmental technologies required to run and maintain the system are outlined in Table 4.1.

Table 4.1: Software and Technology Requirements

Software / Technology	Requirement & Purpose
Operating System	Microsoft Windows 10 / 11 (64-bit), macOS, or Linux
Programming Language	Python 3.10+ (Current Deployment: Python 3.14.0)
Graphical User Interface	Tkinter & ttk (Themed Toolkit with Custom Blush-Pink Boutique Palette)
Database Management System	MySQL Server 8.0 / 8.4 Community Edition
Database Connector	mysql-connector-python 26.7.0 (Native C/Python Client)
Query Language	Standard SQL (DDL, DML, DQL, DCL, Procedural Triggers & Views)
Development Environment	Visual Studio Code / PyCharm / IntelliJ IDEA
Version Control System	Git & GitHub for version management and documentation

4.2 System Requirements for Hardware

As an optimized relational desktop database application, hardware resource demands are moderate:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or equivalent (2.0 GHz or higher).
- **System Memory (RAM):** 4 GB RAM minimum (8 GB recommended for concurrent background processing).
- **Hard Disk Storage:** Minimum 5 GB free storage space for database storage, logs, and application files.
- **Display Resolution:** 1366 x 768 pixels minimum (Optimized for 1920 x 1080 full HD boutique POS displays).
- **Peripherals:** Standard QWERTY keyboard, mouse, or touch-screen pointing device.

4.3 Development Environment

The software architecture is strictly structured into modular functional tiers:

Architecture Pipeline: Tkinter GUI View → Business Logic Layer → Data Access Object (DAO) Layer → mysql.connector → MySQL 8.0 Server

The Presentation View renders high-aesthetic interactive screens. The Business Logic layer enforces client-side date validations and input sanitization. The DAO layer wraps SQL execution with try-except-finally blocks, ensuring explicit commits on success and immediate rollbacks on failure before releasing connections back to the MySQL connection pool.

5. ER Diagram

5.1 Major Entities and Relationships

The Entity-Relationship (ER) diagram serves as the structural foundation of the Tailoring and Boutique Order Management System. It models the core boutique domain entities, attributes, primary identifying keys, foreign key constraints, and inter-entity structural associations. The primary entities modeled include:

- **1. CUSTOMER:** Stores core client identities, codes, names, phone numbers, emails, and home addresses.
- **2. MEASUREMENT_PROFILE:** Maintains exact anatomical tailoring measurements (bust, waist, hip, shoulder, sleeve, length).
- **3. GARMENT_TYPE:** Classifies master garment categories (Kurti, Blouse, Lehenga, Salwar Suit, Gown, Saree Blouse) with base stitching fees.
- **4. DESIGN:** Maintains specific design patterns, cuts, and necklines linked to garment types.
- **5. FABRIC:** Catalogs textile fabrics, materials, colors, and rate per meter.

- **6. FABRIC_STOCK:** Maintains live inventory balances in meters and threshold alert levels.
- **7. ORDER:** Stores order contracts, committed delivery dates, lifecycle statuses, and client instructions.
- **8. ORDER_ITEM:** Associates specific garments, designs, and fabrics within an order, capturing line quantities and rates.
- **9. FABRIC_ISSUE:** Records physical fabric yardage released from stock to tailor workstations.
- **10. TAILOR:** Maintains artisan master tailors, craftsmanship specializations, and active statuses.
- **11. TAILOR_ASSIGNMENT:** Allocates orders to designated tailors with target completion dates.
- **12. TRIAL:** Schedules client trial sessions and fitting evaluations.
- **13. ALTERATION:** Documents fitting corrections, alteration costs, and completion statuses.
- **14. INVOICE:** Consolidates line item totals, alteration additions, and grand billing balances.
- **15. PAYMENT:** Logs monetary payment installments across varied payment instruments.

5.2 ER Diagram Representation

The comprehensive ER diagram illustrating all entities, attributes, primary keys, foreign keys, and cardinalities is displayed in Figure 5.1:

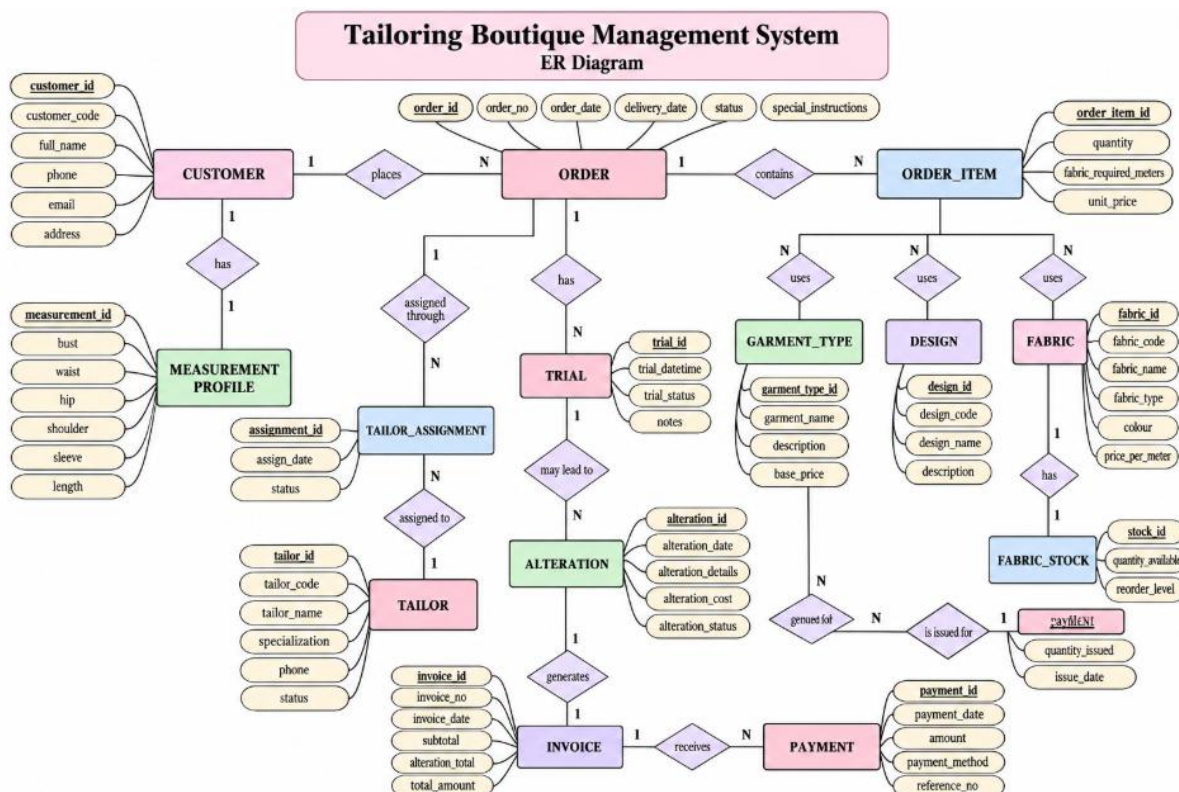


Figure 5.1: Tailoring Boutique Management System ER Diagram

5.3 Important Relationships

The cardinalities and behavioral constraints linking the fifteen database entities are summarized in Table 5.1.

Table 5.1: Important Relationships between Entities

S. No.	Relationship	Cardinality	Description
1	CUSTOMER → MEASUREMENT_PROFILE	1 : 1	Each customer maintains exactly one primary active measurement profile (1:1 with CASCADE delete).
2	CUSTOMER → ORDER	1 : N	One customer can place multiple bespoke boutique orders over time.
3	ORDER → ORDER_ITEM	1 : N	A single order comprises one or more line-level garment items.
4	GARMENT_TYPE → DESIGN	1 : N	One garment category (e.g., Kurti) features multiple style and neckline designs.
5	GARMENT_TYPE → ORDER_ITEM	1 : N	Garment types are selected across multiple order line items.
6	DESIGN → ORDER_ITEM	1 : N	A design pattern is chosen across numerous ordered garments.
7	FABRIC → FABRIC_STOCK	1 : 1	Each fabric catalog entry possesses exactly one stock quantity ledger row.
8	FABRIC → ORDER_ITEM	1 : N	A specific fabric roll can be specified across various order line items.
9	ORDER_ITEM → FABRIC_ISSUE	1 : N	Fabric is issued against specific order items in one or more roll disbursements.
10	FABRIC → FABRIC_ISSUE	1 : N	One fabric roll can be issued multiple times across distinct order items.
11	ORDER → TAILOR_ASSIGNMENT	1 : N	An order can be assigned to multiple tailors for distinct production phases.
12	TAILOR → TAILOR_ASSIGNMENT	1 : N	A master tailor manages multiple simultaneous order assignments.
13	ORDER → TRIAL	1 : N	An order undergoes one or more scheduled customer fitting trials.
14	TAILOR → TRIAL	1 : N	A tailor conducts multiple scheduled trials (unique on tailor_id and trial_datetime).
15	TRIAL → ALTERATION	1 : N	A fitting trial may uncover defects that lead to one or more alteration tasks.
16	ORDER → ALTERATION	1 : N	An order directly tracks all associated alterations and their labor charges.
17	ORDER → INVOICE	1 : 1	An order corresponds to exactly one master billing invoice.

18	INVOICE → PAYMENT	1 : N	An invoice receives multiple payment installments until the balance reaches zero.
----	-------------------	-------	---

6. Relational Schema and Normalization

6.1 Relational Schema

The relational schema converts the conceptual ER diagram into structured relational definitions. Primary keys are explicitly specified as unique table identifiers, and foreign keys link dependent entities to ensure referential integrity. The fifteen relations are defined as follows:

CUSTOMERS : (customer_id [PK], customer_code, full_name, phone, email, address, created_at)

MEASUREMENT_PROFILES : (measurement_id [PK], customer_id [FK to CUSTOMERS], bust_cm, waist_cm, hip_cm, shoulder_cm, sleeve_cm, length_cm, updated_at)

GARMENT_TYPES : (garment_type_id [PK], garment_name, description, base_price)

DESIGNS : (design_id [PK], design_code, design_name, garment_type_id [FK to GARMENT_TYPES], description)

FABRICS : (fabric_id [PK], fabric_code, fabric_name, fabric_type, colour, price_per_meter)

FABRIC_STOCK : (stock_id [PK], fabric_id [FK to FABRICS], quantity_meters, reorder_level, last_updated)

ORDERS : (order_id [PK], order_no, customer_id [FK to CUSTOMERS], order_date, delivery_date, status, special_instructions)

ORDER_ITEMS : (order_item_id [PK], order_id [FK to ORDERS], garment_type_id [FK to GARMENT_TYPES], design_id [FK to DESIGNS], fabric_id [FK to FABRICS], quantity, fabric_required_meters, unit_price)

FABRIC_ISSUES : (issue_id [PK], order_item_id [FK to ORDER_ITEMS], fabric_id [FK to FABRICS], issue_date, quantity_meters)

TAILORS : (tailor_id [PK], tailor_code, tailor_name, specialization, phone, status)

TAILOR_ASSIGNMENTS : (assignment_id [PK], order_id [FK to ORDERS], tailor_id [FK to TAILORS], assigned_on, expected_completion, assignment_status)

TRIALS : (trial_id [PK], order_id [FK to ORDERS], tailor_id [FK to TAILORS], trial_datetime, trial_status, notes)

ALTERATIONS : (alteration_id [PK], order_id [FK to ORDERS], trial_id [FK to TRIALS], alteration_date, alteration_details, alteration_cost, alteration_status)

INVOICES : (invoice_id [PK], invoice_no, order_id [FK to ORDERS], invoice_date, subtotal, alteration_total, total_amount)

PAYMENTS : (payment_id [PK], invoice_id [FK to INVOICES], payment_date, amount, payment_method, reference_no)

6.2 Primary Keys and Foreign Keys

Table 6.1 delineates all identifying primary keys, composite keys, and foreign key linkages enforcing referential integrity across the system.

Table 6.1: Primary Keys and Foreign Keys

Table Name	Primary Key	Foreign Key References & Constraints
CUSTOMERS	customer_id	— (None)
MEASUREMENT_PROFILES	measurement_id	customer_id → CUSTOMERS(customer_id) [ON DELETE CASCADE]
GARMENT_TYPES	garment_type_id	— (None)
DESIGNS	design_id	garment_type_id → GARMENT_TYPES(garment_type_id)
FABRICS	fabric_id	— (None)
FABRIC_STOCK	stock_id	fabric_id → FABRICS(fabric_id) [ON DELETE CASCADE]
ORDERS	order_id	customer_id → CUSTOMERS(customer_id)
ORDER_ITEMS	order_item_id	order_id → ORDERS; garment_type_id → GARMENT_TYPES; design_id → DESIGNS; fabric_id → FABRICS
FABRIC_ISSUES	issue_id	order_item_id → ORDER_ITEMS; fabric_id → FABRICS
TAILORS	tailor_id	— (None)
TAILOR_ASSIGNMENTS	assignment_id	order_id → ORDERS; tailor_id → TAILORS
TRIALS	trial_id	order_id → ORDERS; tailor_id → TAILORS
ALTERATIONS	alteration_id	order_id → ORDERS; trial_id → TRIALS [ON DELETE SET NULL]
INVOICES	invoice_id	order_id → ORDERS(order_id) [ON DELETE CASCADE]
PAYMENTS	payment_id	invoice_id → INVOICES(invoice_id) [ON DELETE CASCADE]

6.3 Normalization

Normalization is the formal mathematical process of decomposing complex relational structures to eliminate redundant data, prevent update/insertion/deletion anomalies, and ensure that every non-key attribute is fully functionally dependent solely on the primary key. The database was

progressively normalized from Unnormalized Form (UNF) through to Third Normal Form (3NF).

Step 1: Unnormalized Form (UNF)

In an unnormalized boutique recording register, an order record contains multi-valued, repeating groups where multiple garment types, designs, fabric materials, and tailor details are concatenated inside individual cells, as shown in Table 6.2.

Table 6.2: Unnormalized Form (UNF) – Sample Order Data

Order No	Customer Name	Phone	Garments	Designs	Fabrics	Tailors
ORD001	Aanya Sharma	9876500001	Kurti, Blouse	Floral Neckline, Princess Cut	Cotton Slub, Raw Silk	Ramesh Kumar
ORD002	Riya Mehta	9876500002	Lehenga	Mirror Work	Raw Silk	Sana Khan
ORD003	Kavya Singh	9876500003	Salwar Suit, Kurti	Straight Cut, Boat Neck	Chanderi, Cotton Slub	Arjun Das

Violation: Columns 'Garments', 'Designs', and 'Fabrics' violate atomicity. A single relational cell cannot store repeating groups.

Step 2: First Normal Form (1NF)

To achieve First Normal Form (1NF), every repeating attribute is flattened so that each cell holds strictly one atomic value, with a composite candidate key (Order No, Garment, Design).

Table 6.3: First Normal Form (1NF) – Sample Order Data

Order No	Customer Name	Phone	Garment	Design	Fabric	Tailor
ORD001	Aanya Sharma	9876500001	Kurti	Floral Neckline	Cotton Slub	Ramesh Kumar
ORD001	Aanya Sharma	9876500001	Blouse	Princess Cut	Raw Silk	Ramesh Kumar
ORD002	Riya Mehta	9876500002	Lehenga	Mirror Work	Raw Silk	Sana Khan
ORD003	Kavya Singh	9876500003	Salwar Suit	Straight Cut	Chanderi	Arjun Das
ORD003	Kavya Singh	9876500003	Kurti	Boat Neck	Cotton Slub	Arjun Das

Condition Achieved: All attributes contain atomic scalar values. However, Customer Name, Phone, and Tailor are repeatedly duplicated for every item, exhibiting partial functional dependencies.

Step 3: Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key, eliminating partial dependencies where an attribute depends only on part of a composite key. Customer Name depends solely on Order No, not on (Order No, Garment). Therefore, the schema is decomposed into ORDERS and ORDER_ITEMS.

Table 6.4: Second Normal Form (2NF) – ORDERS

Order No	Customer Name	Phone	Order Date	Delivery Date	Status
ORD001	Aanya Sharma	9876500001	2026-10-01	2026-10-10	IN_PROGRESS
ORD002	Riya Mehta	9876500002	2026-10-01	2026-10-08	TRIAL
ORD003	Kavya Singh	9876500003	2026-10-02	2026-10-15	PLACED

Table 6.5: Second Normal Form (2NF) – ORDER_ITEMS

Order No	Garment	Design	Fabric	Qty	Price
ORD001	Kurti	Floral Neckline	Cotton Slub	1	1450.00
ORD001	Blouse	Princess Cut	Raw Silk	1	2350.00
ORD002	Lehenga	Mirror Work	Raw Silk	1	8200.00
ORD003	Salwar Suit	Straight Cut	Chanderi	1	3200.00

Condition Achieved: Partial dependencies are resolved. Non-key item attributes depend on the entire item identifier.

Step 4: Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it is in 2NF and contains no transitive dependencies (i.e., non-key attribute X determines non-key attribute Y). In the 2NF table, Customer Phone and Address depend on Customer Name, which in turn depends on Order ID (Order ID → Customer ID → Customer Details). Similarly, Fabric Price depends on Fabric ID, and Garment Base Price depends on Garment Type ID. Decomposing these gives independent master entities:

Table 6.7: Third Normal Form (3NF) – CUSTOMERS

Customer ID	Code	Full Name	Phone	Address
1	C001	Aanya Sharma	9876500001	Banjara Hills, Hyderabad
2	C002	Riya Mehta	9876500002	Kondapur, Hyderabad
3	C003	Kavya Singh	9876500003	Gachibowli, Hyderabad

Table 6.6: Third Normal Form (3NF) – ORDERS

Order ID	Order No	Customer ID [FK]	Order Date	Delivery Date	Status
1	ORD001	1 (C001)	2026-10-01	2026-10-10	IN_PROGRESS
2	ORD002	2 (C002)	2026-10-01	2026-10-08	TRIAL
3	ORD003	3 (C003)	2026-10-02	2026-10-15	PLACED

Table 6.8: Third Normal Form (3NF) – ORDER_ITEMS

Item ID	Order ID [FK]	Garment [FK]	Design [FK]	Fabric [FK]	Qty	Price
1	1	1 (Kurti)	1 (Floral Neck)	1 (Cotton Slub)	1	1450.00
2	1	2 (Blouse)	2 (Princess Cut)	2 (Raw Silk)	1	2350.00
3	2	3 (Lehenga)	3 (Mirror Work)	2 (Raw Silk)	1	8200.00

Table 6.9: Third Normal Form (3NF) – FABRICS

Fabric ID	Code	Fabric Name	Type	Colour	Price/Meter
1	F001	Cotton Slub	Natural Cotton	Blush Pink	420.00
2	F002	Raw Silk	Pure Silk	Royal Maroon	850.00
3	F003	Georgette	Synthetic	Midnight Black	620.00

Table 6.10: Third Normal Form (3NF) – TAILORS

Tailor ID	Code	Tailor Name	Specialization	Phone	Status
1	T001	Ramesh Kumar	Blouse Specialist	9000000001	ACTIVE
2	T002	Sana Khan	Lehenga Specialist	9000000002	ACTIVE
3	T003	Priya Nair	Western Gowns	9000000003	ACTIVE

Conclusion of Normalization: The complete relational design strictly satisfies Third Normal Form (3NF). Every non-key attribute is directly, non-transitively dependent on the key, the whole key, and nothing but the key.

7. Data Dictionary

The Data Dictionary provides an exhaustive structural specification of all database entities, enumerating attribute names, descriptions, data types, lengths, and constraints (Primary Key, Foreign Key, UNIQUE, NOT NULL, CHECK, DEFAULT).

7.1 CUSTOMERS

Table 7.1: CUSTOMERS Table Data Dictionary

Attribute	Description	Data Type	Constraints
customer_id	Surrogate unique customer identifier	INT	PRIMARY KEY, AUTO_INCREMENT
customer_code	Business customer code (e.g., C001)	VARCHAR(10)	NOT NULL, UNIQUE
full_name	Complete customer name	VARCHAR(100)	NOT NULL
phone	Primary contact telephone number	VARCHAR(15)	NOT NULL, UNIQUE
email	Electronic mail address	VARCHAR(120)	UNIQUE, NULLABLE
address	Residential or delivery address	VARCHAR(255)	NULLABLE
created_at	Record registration timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

7.2 MEASUREMENT_PROFILES

Table 7.2: MEASUREMENT_PROFILES Table Data Dictionary

Attribute	Description	Data Type	Constraints
measurement_id	Measurement profile record key	INT	PRIMARY KEY, AUTO_INCREMENT
customer_id	Associated client reference	INT	FK → customers(customer_id), UNIQUE, ON DELETE CASCADE
bust_cm	Bust girth dimension in cm	DECIMAL(5,2)	CHECK (bust_cm > 0)
waist_cm	Waist circumference in cm	DECIMAL(5,2)	CHECK (waist_cm > 0)
hip_cm	Hip circumference in cm	DECIMAL(5,2)	CHECK (hip_cm > 0)
shoulder_cm	Shoulder tip-to-tip width in cm	DECIMAL(5,2)	CHECK (shoulder_cm > 0)
sleeve_cm	Sleeve length in cm	DECIMAL(5,2)	CHECK (sleeve_cm > 0)
length_cm	Full garment length in cm	DECIMAL(5,2)	CHECK (length_cm > 0)
updated_at	Last measurement adjustment timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE

7.3 GARMENT_TYPES

Table 7.3: GARMENT_TYPES Table Data Dictionary

Attribute	Description	Data Type	Constraints
garment_type_id	Garment category identifier	INT	PRIMARY KEY, AUTO_INCREMENT
garment_name	Category name (Kurti, Blouse, Lehenga)	VARCHAR(60)	NOT NULL, UNIQUE
description	Styling specifications and guidelines	VARCHAR(255)	NULLABLE
base_price	Baseline tailoring labor cost	DECIMAL(10,2)	NOT NULL, CHECK (base_price >= 0)

7.4 DESIGNS

Table 7.4: DESIGNS Table Data Dictionary

Attribute	Description	Data Type	Constraints
design_id	Design pattern identifier	INT	PRIMARY KEY, AUTO_INCREMENT
design_code	Alphanumeric pattern code (D001)	VARCHAR(15)	NOT NULL, UNIQUE
design_name	Name of design cut or embroidery style	VARCHAR(100)	NOT NULL
garment_type_id	Applicable garment silhouette	INT	FK → garment_types(garment_type_id)
description	Detailed embroidery and neckline notes	VARCHAR(255)	NULLABLE

7.5 FABRICS

Table 7.5: FABRICS Table Data Dictionary

Attribute	Description	Data Type	Constraints
fabric_id	Fabric catalog identifier	INT	PRIMARY KEY, AUTO_INCREMENT
fabric_code	Inventory code for textile roll (F001)	VARCHAR(15)	NOT NULL, UNIQUE
fabric_name	Commercially recognized fabric name	VARCHAR(100)	NOT NULL
fabric_type	Textile material composition	VARCHAR(50)	NOT NULL
colour	Fabric dye shade description	VARCHAR(40)	NULLABLE
price_per_meter	Boutique retail price per linear meter	DECIMAL(10,2)	NOT NULL, CHECK (price_per_meter >= 0)

7.6 FABRIC_STOCK

Table 7.6: FABRIC_STOCK Table Data Dictionary

Attribute	Description	Data Type	Constraints
stock_id	Inventory stock ledger identifier	INT	PRIMARY KEY, AUTO_INCREMENT
fabric_id	Referenced fabric material	INT	FK → fabrics(fabric_id), UNIQUE, ON DELETE CASCADE
quantity_meters	Available physically measured meters	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (>= 0)
reorder_level	Automated low-stock threshold trigger	DECIMAL(10,2)	NOT NULL, DEFAULT 5.0, CHECK (>= 0)
last_updated	Timestamp of latest inventory balance update	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE

7.7 ORDERS

Table 7.7: ORDERS Table Data Dictionary

Attribute	Description	Data Type	Constraints
order_id	Surrogate order identifier	INT	PRIMARY KEY, AUTO_INCREMENT
order_no	Customer-facing tracking code (ORD001)	VARCHAR(20)	NOT NULL, UNIQUE
customer_id	Ordering client reference	INT	NOT NULL, FK → customers(customer_id)
order_date	Booking date	DATE	NOT NULL, DEFAULT CURRENT_DATE
delivery_date	Committed handover date	DATE	NOT NULL, CHECK (delivery_date >= order_date)
status	Lifecycle state in production	ENUM	ENUM('PLACED','IN_PROGRESS','TRIAL','ALTERATION','READY','DELIVERED','CANCELLED')

special_instructions	Custom client tailoring requests	VARCHAR(500)	NULLABLE
----------------------	----------------------------------	--------------	----------

7.8 ORDER_ITEMS

Table 7.8: ORDER_ITEMS Table Data Dictionary

Attribute	Description	Data Type	Constraints
order_item_id	Line-item surrogate identifier	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	Parent order reference	INT	NOT NULL, FK → orders(order_id) ON DELETE CASCADE
garment_type_id	Selected garment category	INT	NOT NULL, FK → garment_types(garment_type_id)
design_id	Selected neckline / pattern design	INT	NULLABLE, FK → designs(design_id)
fabric_id	Chosen fabric material	INT	NOT NULL, FK → fabrics(fabric_id)
quantity	Garment pieces ordered	INT	NOT NULL, DEFAULT 1, CHECK (quantity > 0)
fabric_required_meters	Required yardage for garment cutting	DECIMAL(10,2)	NOT NULL, CHECK (> 0)
unit_price	Agreed price per garment piece	DECIMAL(10,2)	NOT NULL, CHECK (unit_price >= 0)

7.9 FABRIC_ISSUES

Table 7.9: FABRIC_ISSUES Table Data Dictionary

Attribute	Description	Data Type	Constraints
issue_id	Fabric disbursement transaction key	INT	PRIMARY KEY, AUTO_INCREMENT
order_item_id	Assigned order line garment	INT	NOT NULL, FK → order_items(order_item_id) CASCADE
fabric_id	Deducted fabric roll	INT	NOT NULL, FK → fabrics(fabric_id)
issue_date	Date material handed to cutter	DATE	NOT NULL, DEFAULT CURRENT_DATE

quantity_meters	Exact meters released to tailor	DECIMAL(10,2)	NOT NULL, CHECK (quantity_meters > 0)
-----------------	---------------------------------	---------------	---------------------------------------

7.10 TAILORS

Table 7.10: TAILORS Table Data Dictionary

Attribute	Description	Data Type	Constraints
tailor_id	Artisan tailor identifier	INT	PRIMARY KEY, AUTO_INCREMENT
tailor_code	Staff employee code (T001)	VARCHAR(15)	NOT NULL, UNIQUE
tailor_name	Full name of tailor	VARCHAR(100)	NOT NULL
specialization	Core craft (Blouse, Lehenga, Gowns)	VARCHAR(80)	NULLABLE
phone	Personal mobile contact	VARCHAR(15)	UNIQUE, NULLABLE
status	Current employment duty state	ENUM	ENUM('ACTIVE','INACTIVE'), DEFAULT 'ACTIVE'

7.11 TAILOR_ASSIGNMENTS

Table 7.11: TAILOR_ASSIGNMENTS Table Data Dictionary

Attribute	Description	Data Type	Constraints
assignment_id	Order-to-tailor assignment key	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	Allocated order contract	INT	NOT NULL, FK → orders(order_id) CASCADE
tailor_id	Designated master artisan	INT	NOT NULL, FK → tailors(tailor_id)
assigned_on	Date work delegated	DATE	NOT NULL, DEFAULT CURRENT_DATE
expected_completion	Target stitching deadline	DATE	CHECK (expected_completion >= assigned_on)
assignment_status	Task progress state	ENUM	ENUM('ASSIGNED','IN_PROGRESS','COMPLETED'), DEFAULT 'ASSIGNED'

7.12 TRIALS

Table 7.12: TRIALS Table Data Dictionary

Attribute	Description	Data Type	Constraints
trial_id	Fitting appointment record key	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	Order subject to fitting review	INT	NOT NULL, FK → orders(order_id) CASCADE
tailor_id	Tailor attending fitting session	INT	NOT NULL, FK → tailors(tailor_id)
trial_datetime	Exact date and time of trial	DATETIME	NOT NULL, UNIQUE(tailor_id, trial_datetime)
trial_status	Session outcome	ENUM	ENUM('SCHEDULED','COMPLETED','RESCHEDULED','CANCELLED')
notes	Fitter observations and adjustment notes	VARCHAR(255)	NULLABLE

7.13 ALTERATIONS

Table 7.13: ALTERATIONS Table Data Dictionary

Attribute	Description	Data Type	Constraints
alteration_id	Post-trial alteration identifier	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	Target order being modified	INT	NOT NULL, FK → orders(order_id) CASCADE
trial_id	Preceding trial session origin	INT	NULLABLE, FK → trials(trial_id) SET NULL
alteration_date	Date alteration logged	DATE	NOT NULL, DEFAULT CURRENT_DATE
alteration_details	Specific rework instructions	VARCHAR(500)	NOT NULL
alteration_cost	Supplementary alteration charge	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (>= 0)
alteration_status	Current rework status	ENUM	ENUM('PENDING','IN_PROGRESS','COMPLETED'), DEFAULT 'PENDING'

7.14 INVOICES

Table 7.14: INVOICES Table Data Dictionary

Attribute	Description	Data Type	Constraints
invoice_id	Master billing invoice identifier	INT	PRIMARY KEY, AUTO_INCREMENT
invoice_no	Official accounting serial (INV001)	VARCHAR(20)	NOT NULL, UNIQUE
order_id	Associated boutique order	INT	NOT NULL, UNIQUE, FK → orders(order_id) CASCADE
invoice_date	Billing invoice creation date	DATE	NOT NULL, DEFAULT CURRENT_DATE
subtotal	Cumulative order items base price	DECIMAL(10,2)	NOT NULL, CHECK (subtotal >= 0)
alteration_total	Cumulative alteration rework charges	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (>= 0)
total_amount	Final invoice amount (subtotal + alt)	DECIMAL(10,2)	NOT NULL, CHECK (total_amount = subtotal + alteration_total)

7.15 PAYMENTS

Table 7.15: PAYMENTS Table Data Dictionary

Attribute	Description	Data Type	Constraints
payment_id	Payment transaction identifier	INT	PRIMARY KEY, AUTO_INCREMENT
invoice_id	Target billing invoice reference	INT	NOT NULL, FK → invoices(invoice_id) CASCADE
payment_date	Date money received	DATE	NOT NULL, DEFAULT CURRENT_DATE
amount	Monetary value transacted	DECIMAL(10,2)	NOT NULL, CHECK (amount > 0)
payment_method	Instrument of transaction	ENUM	ENUM('CASH','UPI','CARD','BANK_TRANSFER')
reference_no	Transaction UTR or receipt reference	VARCHAR(40)	UNIQUE, NULLABLE

8. SQL Commands Used

This section illustrates the Data Definition Language (DDL) and Data Manipulation Language (DML) statements utilized to construct, constrain, and manipulate the database schema.

8.1 DDL Commands

Data Definition Language commands establish database structure, define relations, build indexes, declare views, and configure triggers:

- **1. Database Creation:** CREATE DATABASE IF NOT EXISTS tailoring_boutique;
- **2. Table Creation:** CREATE TABLE customers (customer_id INT AUTO_INCREMENT PRIMARY KEY, customer_code VARCHAR(10) NOT NULL UNIQUE, full_name VARCHAR(100) NOT NULL, phone VARCHAR(15) NOT NULL UNIQUE, email VARCHAR(120) UNIQUE, address VARCHAR(255), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
- **3. Check Constraints:** CONSTRAINT chk_delivery_date CHECK (delivery_date >= order_date)
- **4. Multi-Column Unique Constraint:** UNIQUE(tailor_id, trial_datetime) preventing conflicting appointment bookings.
- **5. Performance Indexing:** CREATE INDEX idx_orders_delivery_status ON orders(delivery_date, status);
- **6. View Creation:** CREATE OR REPLACE VIEW v_delivery_due AS SELECT o.order_no, c.full_name AS customer, o.delivery_date, o.status FROM orders o JOIN customers c ON c.customer_id = o.customer_id WHERE o.delivery_date <= DATE_ADD(CURRENT_DATE, INTERVAL 7 DAY) AND o.status NOT IN ('DELIVERED','CANCELLED');
- **7. Trigger Creation:** CREATE TRIGGER trg_fabric_issue_before_insert BEFORE INSERT ON fabric_issues ... SIGNAL SQLSTATE '45000' ...

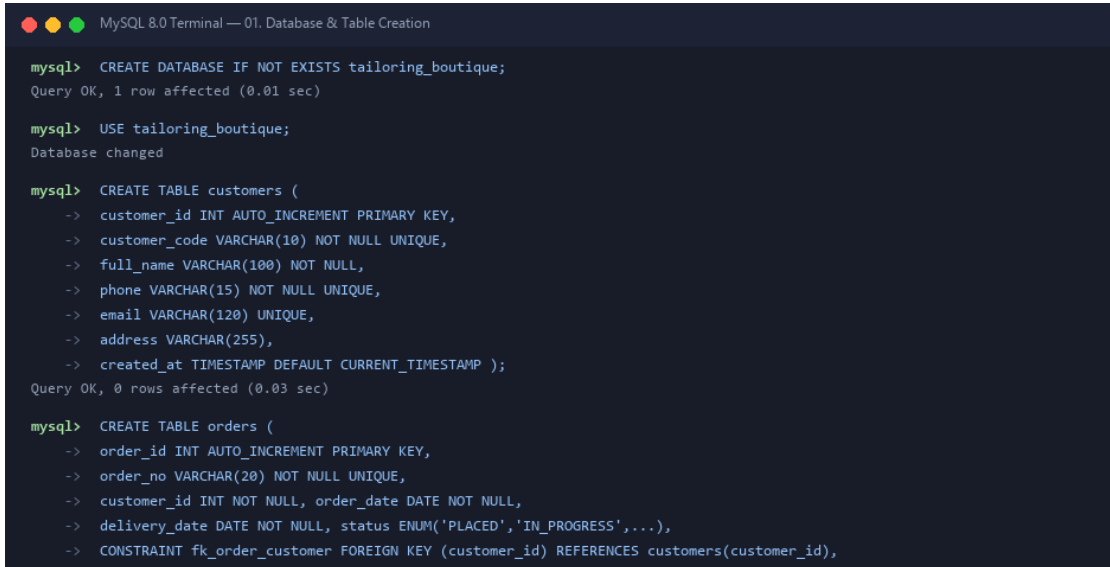
8.2 DML Commands

Data Manipulation Language commands insert, modify, remove, and retrieve records across the normalized tables:

- **1. INSERT Statement:** INSERT INTO customers (customer_code, full_name, phone, email, address) VALUES ('C001', 'Aanya Sharma', '9876500001', 'aanya@example.com', 'Banjara Hills, Hyderabad');
- **2. UPDATE Statement:** UPDATE orders SET status = 'IN_PROGRESS' WHERE order_no = 'ORD001';
- **3. DELETE Statement:** DELETE FROM alterations WHERE alteration_id = 6;
- **4. SELECT Statement:** SELECT c.full_name, o.order_no, o.status, i.total_amount FROM customers c JOIN orders o ON o.customer_id = c.customer_id LEFT JOIN invoices i ON i.order_id = o.order_id;

8.3 Sample Output

Visual execution confirmations from the MySQL database terminal environment are documented in Figures 8.1 and 8.2:



```
mysql> CREATE DATABASE IF NOT EXISTS tailoring_boutique;
Query OK, 1 row affected (0.01 sec)

mysql> USE tailoring_boutique;
Database changed

mysql> CREATE TABLE customers (
  -> customer_id INT AUTO_INCREMENT PRIMARY KEY,
  -> customer_code VARCHAR(10) NOT NULL UNIQUE,
  -> full_name VARCHAR(100) NOT NULL,
  -> phone VARCHAR(15) NOT NULL UNIQUE,
  -> email VARCHAR(120) UNIQUE,
  -> address VARCHAR(255),
  -> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE orders (
  -> order_id INT AUTO_INCREMENT PRIMARY KEY,
  -> order_no VARCHAR(20) NOT NULL UNIQUE,
  -> customer_id INT NOT NULL, order_date DATE NOT NULL,
  -> delivery_date DATE NOT NULL, status ENUM('PLACED','IN_PROGRESS',...),
  -> CONSTRAINT fk_order_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
```

Figure 8.1: Database and Table Creation



```
mysql> INSERT INTO orders (order_no, customer_id, order_date, delivery_date, status, special_instructions)
  -> VALUES ('ORD007', 1, '2026-10-07', '2026-10-16', 'PLACED', 'Gold zari embroidery on collar');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO fabric_issues (order_item_id, fabric_id, issue_date, quantity_meters)
  -> VALUES (7, 2, '2026-10-07', 3.50);
Query OK, 1 row affected (0.01 sec) [Fabric stock updated via trg_fabric_issue_after_insert]

mysql> UPDATE orders SET status = 'IN_PROGRESS' WHERE order_no = 'ORD007';
Query OK, 1 row affected (0.01 sec) - Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT o.order_no, c.full_name, o.delivery_date, o.status FROM orders o
  -> JOIN customers c ON c.customer_id = o.customer_id WHERE o.order_no = 'ORD007';
+-----+-----+-----+-----+
| order_no | full_name | delivery_date | status |
+-----+-----+-----+-----+
| ORD007 | Aanya Sharma | 2026-10-16 | IN_PROGRESS |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Figure 8.2: DML Operation and Output

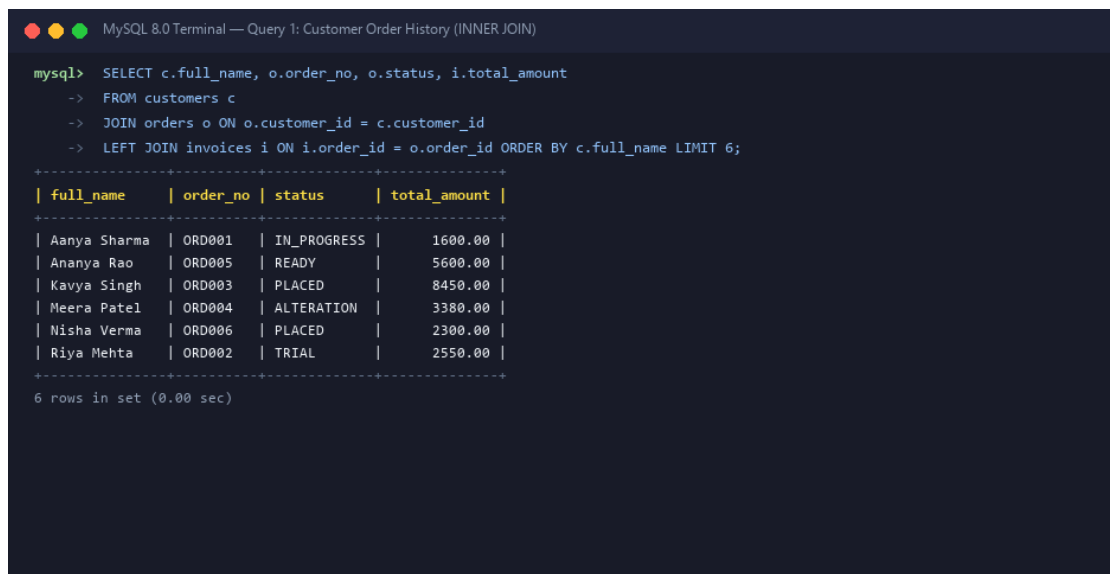
9. Queries with Outputs

The project incorporates advanced SQL query constructs demonstrating joins, multi-table joins, aggregations, filtering, nested subqueries, conditional logic, and database views based on live MySQL execution.

9.1 Query 1 – Customer Order History

SQL Query: SELECT c.full_name, o.order_no, o.status, i.total_amount FROM customers c JOIN orders o ON o.customer_id = c.customer_id LEFT JOIN invoices i ON i.order_id = o.order_id ORDER BY c.full_name;

Concept: Inner join connecting customers to orders combined with a left outer join to invoices to ensure order records without finalized invoices are still retrieved.



```
mysql> SELECT c.full_name, o.order_no, o.status, i.total_amount
-> FROM customers c
-> JOIN orders o ON o.customer_id = c.customer_id
-> LEFT JOIN invoices i ON i.order_id = o.order_id ORDER BY c.full_name LIMIT 6;

+-----+
| full_name | order_no | status | total_amount |
+-----+
| Anya Sharma | ORD001 | IN_PROGRESS | 1600.00 |
| Ananya Rao | ORD005 | READY | 5600.00 |
| Kavya Singh | ORD003 | PLACED | 8450.00 |
| Meera Patel | ORD004 | ALTERATION | 3380.00 |
| Nisha Verma | ORD006 | PLACED | 2300.00 |
| Riya Mehta | ORD002 | TRIAL | 2550.00 |
+-----+

6 rows in set (0.00 sec)
```

Figure 9.1: Query 1 Execution – Customer Order History

9.2 Query 2 – Orders with Garment Types and Designs

SQL Query: SELECT o.order_no, c.full_name, gt.garment_name, d.design_name, f.fabric_name, oi.unit_price FROM orders o JOIN customers c ON c.customer_id = o.customer_id JOIN order_items oi ON oi.order_id = o.order_id JOIN garment_types gt ON gt.garment_type_id = oi.garment_type_id LEFT JOIN designs d ON d.design_id = oi.design_id JOIN fabrics f ON f.fabric_id = oi.fabric_id;

Concept: Multi-table join linking five relational tables to reconstruct the complete bespoke specification for every line item.

9.3 Query 3 – Orders Due for Delivery Within 7 Days

SQL Query: SELECT order_no, customer, delivery_date, status FROM v_delivery_due ORDER BY delivery_date;

Concept: Date arithmetic and temporal interval filtering (delivery_date <= CURRENT_DATE + INTERVAL 7 DAY) encapsulated within a clean database view.

```
MySQL 8.0 Terminal — Query 3: Orders Due Soon (View + Date Interval)

mysql> SELECT order_no, customer, delivery_date, status
-> FROM v_delivery_due ORDER BY delivery_date;

+-----+-----+-----+-----+
| order_no | customer | delivery_date | status |
+-----+-----+-----+-----+
| ORD002 | Riya Mehta | 2026-10-08 | TRIAL |
| ORD001 | Aanya Sharma | 2026-10-10 | IN_PROGRESS |
| ORD004 | Meera Patel | 2026-10-12 | ALTERATION |
| ORD003 | Kavya Singh | 2026-10-15 | PLACED |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Figure 9.3: Query 3 Execution – Orders Due for Delivery Within 7 Days

9.4 Query 4 – Tailor Workload and Assignment Breakdown

SQL Query: SELECT t.tailor_name, COUNT(ta.assignment_id) AS assigned_orders, SUM(CASE WHEN ta.assignment_status = 'COMPLETED' THEN 1 ELSE 0 END) AS completed, SUM(CASE WHEN ta.assignment_status <> 'COMPLETED' THEN 1 ELSE 0 END) AS active_load FROM tailors t LEFT JOIN tailor_assignments ta ON ta.tailor_id = t.tailor_id GROUP BY t.tailor_id, t.tailor_name ORDER BY active_load DESC;

Concept: LEFT OUTER JOIN with conditional aggregation (SUM(CASE WHEN...)) and GROUP BY to evaluate artisan capacity without omitting tailors with zero current tasks.

```
MySQL 8.0 Terminal — Query 4: Tailor Workload & Assignment Distribution

mysql> SELECT t.tailor_name, COUNT(ta.assignment_id) AS assigned_orders,
-> SUM(CASE WHEN ta.assignment_status='COMPLETED' THEN 1 ELSE 0 END) AS completed,
-> SUM(CASE WHEN ta.assignment_status<>'COMPLETED' THEN 1 ELSE 0 END) AS active_load
-> FROM tailors t LEFT JOIN tailor_assignments ta ON ta.tailor_id=t.tailor_id
-> GROUP BY t.tailor_id, t.tailor_name ORDER BY active_load DESC;

+-----+-----+-----+-----+
| tailor_name | assigned_orders | completed | active_load |
+-----+-----+-----+-----+
| Ramesh Kumar | 1 | 0 | 1 |
| Arjun Das | 1 | 0 | 1 |
| Neha Joshi | 1 | 0 | 1 |
| Vikram Rao | 1 | 0 | 1 |
| Sana Khan | 1 | 1 | 0 |
| Priya Nair | 1 | 1 | 0 |
+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

Figure 9.4: Query 4 Execution – Tailor Workload and Assignment Breakdown

9.5 Query 5 – Fabric Reorder Threshold Alert

SQL Query: SELECT f.fabric_code, f.fabric_name, fs.quantity_meters, fs.reorder_level FROM fabrics f JOIN fabric_stock fs ON fs.fabric_id = f.fabric_id WHERE fs.quantity_meters <= fs.reorder_level;

Concept: Relational join with numeric inequality filtering for automated inventory reordering triggers.

9.6 Query 6 – Order Status Distribution Across Workflow

SQL Query: SELECT status, COUNT(*) AS total_orders FROM orders GROUP BY status ORDER BY total_orders DESC;

Concept: Aggregate COUNT function with GROUP BY clause to identify workflow bottlenecks across boutique production stages.

9.7 Query 7 – Total Alteration Expenditure Analysis

SQL Query: SELECT SUM(alteration_cost) AS total_alteration_cost, AVG(alteration_cost) AS avg_alteration_cost, COUNT(*) AS total_alterations FROM alterations;

Concept: Aggregate financial calculation functions (SUM, AVG, COUNT) evaluating labor overheads incurred in garment rework.

9.8 Query 8 – Maximum Order Value (Nested Subquery)

SQL Query: SELECT o.order_no, c.full_name, i.total_amount FROM invoices i JOIN orders o ON o.order_id = i.order_id JOIN customers c ON c.customer_id = o.customer_id WHERE i.total_amount = (SELECT MAX(total_amount) FROM invoices);

Concept: Nested subquery evaluating a scalar aggregate comparison within the outer WHERE filter.



```
MySQL 8.0 Terminal — Query 8: Maximum Order Value (Nested Subquery)

mysql> SELECT o.order_no, c.full_name, i.total_amount
-> FROM invoices i
-> JOIN orders o ON o.order_id = i.order_id
-> JOIN customers c ON c.customer_id = o.customer_id
-> WHERE i.total_amount = (SELECT MAX(total_amount) FROM invoices);

+-----+
| order_no | full_name | total_amount |
+-----+
| ORD003   | Kavya Singh | 8450.00      |
+-----+
1 row in set (0.01 sec)
```

Figure 9.8: Query 8 Execution – Maximum Order Value (Nested Subquery)

9.9 Query 9 – Orders with Tailor and Fitting Stage Details

SQL Query: SELECT o.order_no, c.full_name, t.tailor_name, tr.trial_datetime, tr.trial_status
FROM orders o JOIN customers c ON c.customer_id = o.customer_id LEFT JOIN trials tr ON
tr.order_id = o.order_id LEFT JOIN tailors t ON t.tailor_id = tr.tailor_id;

Concept: Multiple outer joins consolidating multi-phase scheduling details for client trial follow-ups.

9.10 Query 10 – Invoices Payment Collection and Outstanding Balance

SQL Query: SELECT i.invoice_no, i.total_amount, COALESCE(SUM(p.amount), 0) AS paid,
(i.total_amount - COALESCE(SUM(p.amount), 0)) AS balance FROM invoices i LEFT JOIN
payments p ON p.invoice_id = i.invoice_id GROUP BY i.invoice_id, i.invoice_no,
i.total_amount;

Concept: LEFT JOIN with COALESCE to handle NULL payments and arithmetic expression calculation for outstanding balances.

```

MySQL 8.0 Terminal — Query 10: Invoices Payment Collection & Balance

mysql> SELECT i.invoice_no, i.total_amount,
-> COALESCE(SUM(p.amount),0) AS paid,
-> i.total_amount - COALESCE(SUM(p.amount),0) AS balance
-> FROM invoices i
-> LEFT JOIN payments p ON p.invoice_id = i.invoice_id
-> GROUP BY i.invoice_id, i.invoice_no, i.total_amount;

+-----+
| invoice_no | total_amount | paid   | balance |
+-----+
| INV001    | 1600.00     | 800.00 | 800.00  |
| INV002    | 2550.00     | 1000.00 | 1550.00 |
| INV003    | 8450.00     | 4000.00 | 4450.00 |
| INV004    | 3380.00     | 1500.00 | 1880.00 |
| INV005    | 5600.00     | 5600.00 | 0.00    |
| INV006    | 2300.00     | 1000.00 | 1300.00 |
+-----+
6 rows in set (0.01 sec)

```

Figure 9.10: Query 10 Execution – Invoices Payment Collection and Balance

9.11 Query 11 – Fabric Utilization per Garment Order

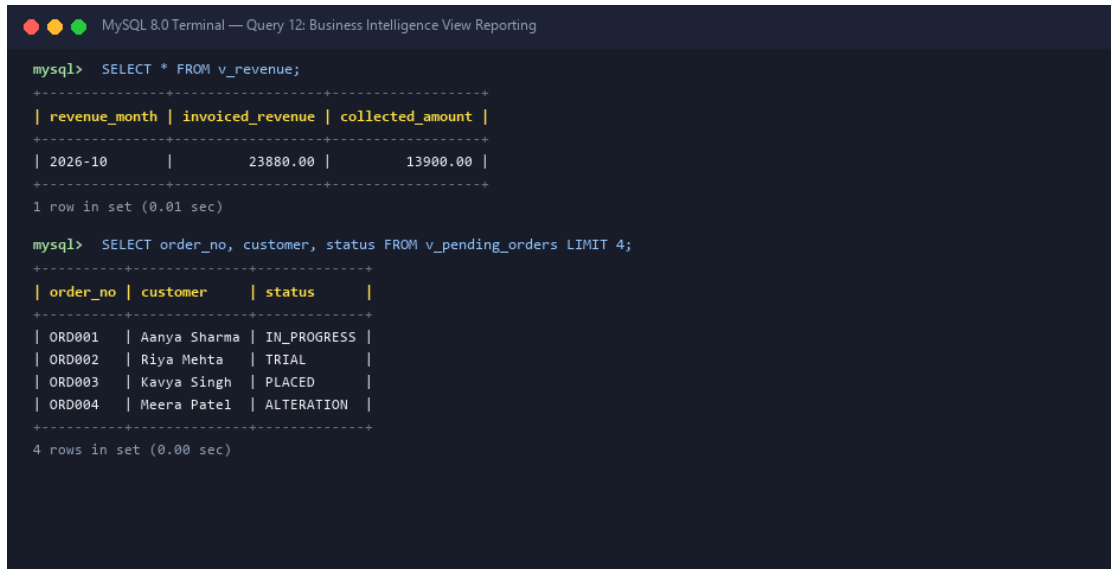
SQL Query: SELECT f.fabric_name, SUM(fi.quantity_meters) AS meters_issued, COUNT(DISTINCT fi.order_item_id) AS items_cut FROM fabrics f JOIN fabric_issues fi ON fi.fabric_id = f.fabric_id GROUP BY f.fabric_id, f.fabric_name;

Concept: Grouped aggregation tracking physical yardage consumption and distinct item production rates.

9.12 Query 12 – View-based Business Intelligence Reporting

SQL Query: SELECT * FROM v_revenue; SELECT order_no, customer, status FROM v_pending_orders LIMIT 4;

Concept: Database VIEW abstraction providing reusable, high-performance executive reporting queries.



```
MySQL 8.0 Terminal — Query 12: Business Intelligence View Reporting

mysql> SELECT * FROM v_revenue;
+-----+-----+-----+
| revenue_month | invoiced_revenue | collected_amount |
+-----+-----+-----+
| 2026-10       | 23880.00         | 13900.00         |
+-----+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT order_no, customer, status FROM v_pending_orders LIMIT 4;
+-----+-----+-----+
| order_no | customer   | status   |
+-----+-----+-----+
| ORD001   | Aanya Sharma | IN_PROGRESS |
| ORD002   | Riya Mehta  | TRIAL     |
| ORD003   | Kavya Singh | PLACED     |
| ORD004   | Meera Patel  | ALTERATION |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Figure 9.12: Query 12 Execution – View-based Business Intelligence Reporting

10. UI Design and Screenshots

The application provides a desktop graphical user interface crafted in Python Tkinter utilizing ttk widgets styled in a tailored blush-pink and clean-white boutique visual theme (#FFF8FB canvas, #FFF0F5 navigation bar, #D96D91 rose buttons). The system connects live to the MySQL database via mysql.connector.

10.1 Application Launch & Aesthetic Design

The interface incorporates a persistent left navigation sidebar featuring twelve primary functional modules, real-time database connection heartbeat monitoring, high-contrast KPI metric cards, searchable data tables with column sortability, and validated pop-up dialogs.

10.2 Boutique Overview Dashboard

The main landing screen presents executive summaries including Total Clients, Active Orders in pipeline, Invoiced Revenue, and Payment Collections, accompanied by an urgent deliveries panel.

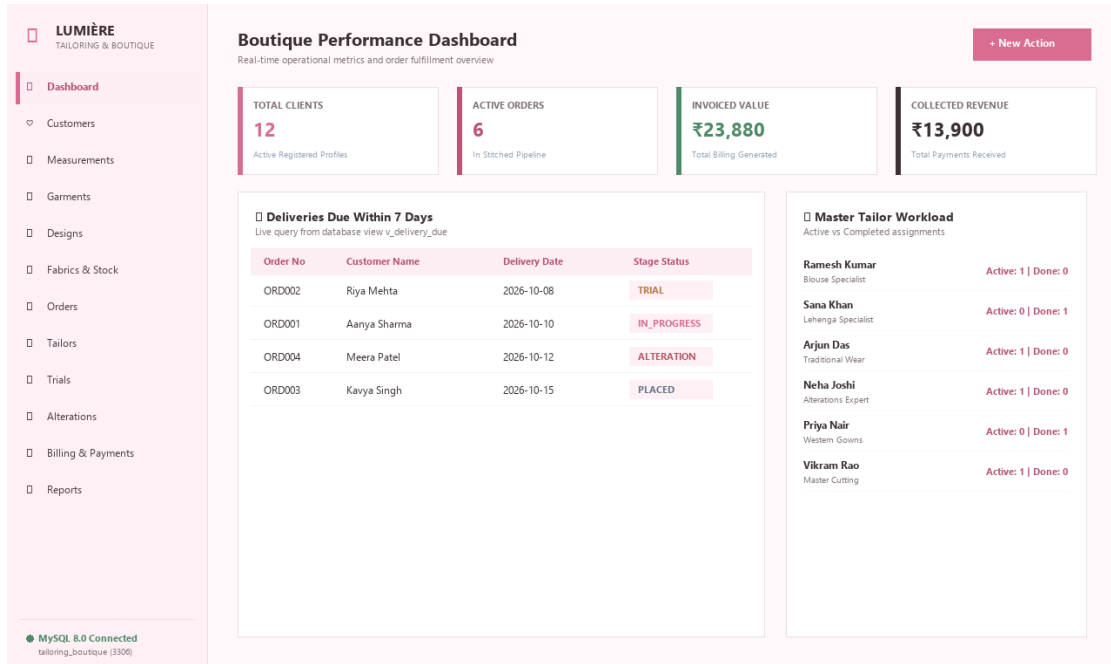


Figure 10.1: Boutique Performance Dashboard Screen

10.3 Customer Management Screen

Permits front-desk staff to register new customers, search client profiles by name/phone/code, view customer history, and manage contact coordinates.

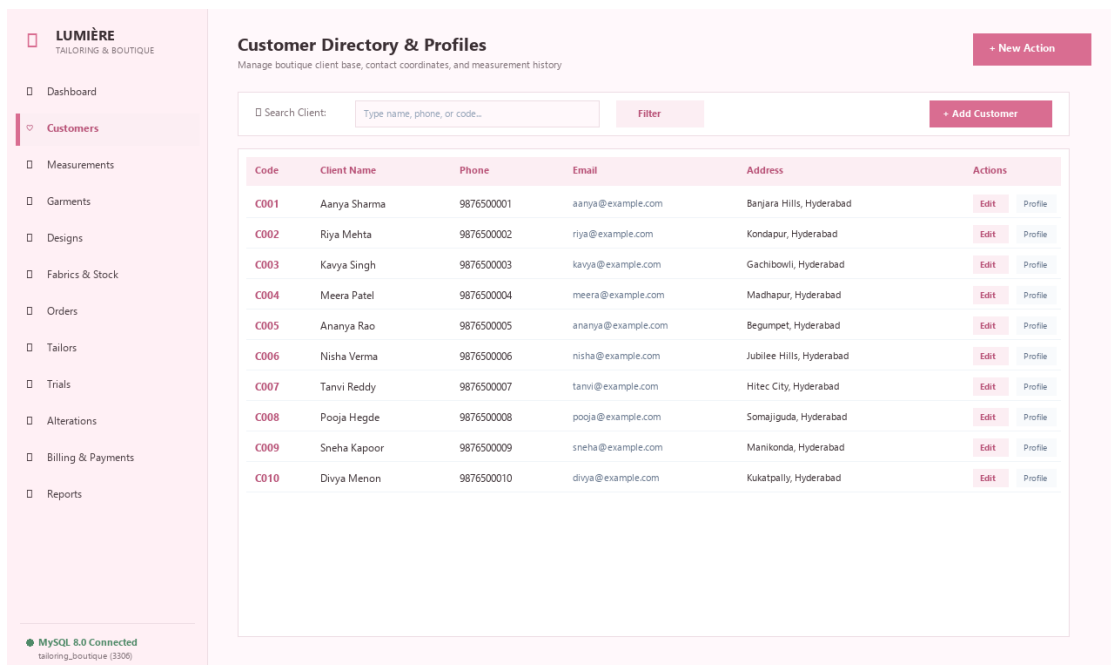


Figure 10.2: Customer Management Directory Screen

10.4 Measurement Profiles Screen

Provides tailored input forms for anatomical specifications (bust, waist, hip, shoulder width, sleeve length, garment length) bound to customer profiles.

LUMIÈRE
TAILORING & BOUTIQUE

- Dashboard
- Customers
- Measurements**
- Garments
- Designs
- Fabrics & Stock
- Orders
- Tailors
- Trials
- Alterations
- Billing & Payments
- Reports

Bespoke Measurement Profiles
Record and manage customized anatomical dimensions for precision tailoring

+ New Action

Client Measurement Form
Select client and update metrics (cm)

Customer Selection
C001 — Aanya Sharma

Bust (cm)
86.00

Waist (cm)
70.00

Hip (cm)
92.00

Shoulder Width (cm)
36.00

Sleeve Length (cm)
56.00

Garment Length (cm)
42.00

Save Measurements

Active Measurements Repository

Client	Bust	Waist	Hip	Shoulder	Sleeve	Length
Aanya Sharma	86.0	70.0	92.0	36.0	56.0	42.0
Riya Mehta	90.0	74.0	96.0	37.0	57.0	44.0
Kavya Singh	88.0	72.0	94.0	36.0	55.0	43.0
Meera Patel	92.0	78.0	100.0	38.0	58.0	45.0
Ananya Rao	84.0	68.0	90.0	35.0	54.0	41.0
Nisha Verma	94.0	80.0	102.0	39.0	59.0	46.0

MySQL 8.0 Connected
tailoring_boutique (3306)

Figure 10.3: Bespoke Measurement Profiles Screen

10.5 Fabrics & Stock Inventory Screen

Displays current stock levels in meters, fabric composition, unit prices, and threshold alerts for reordering.

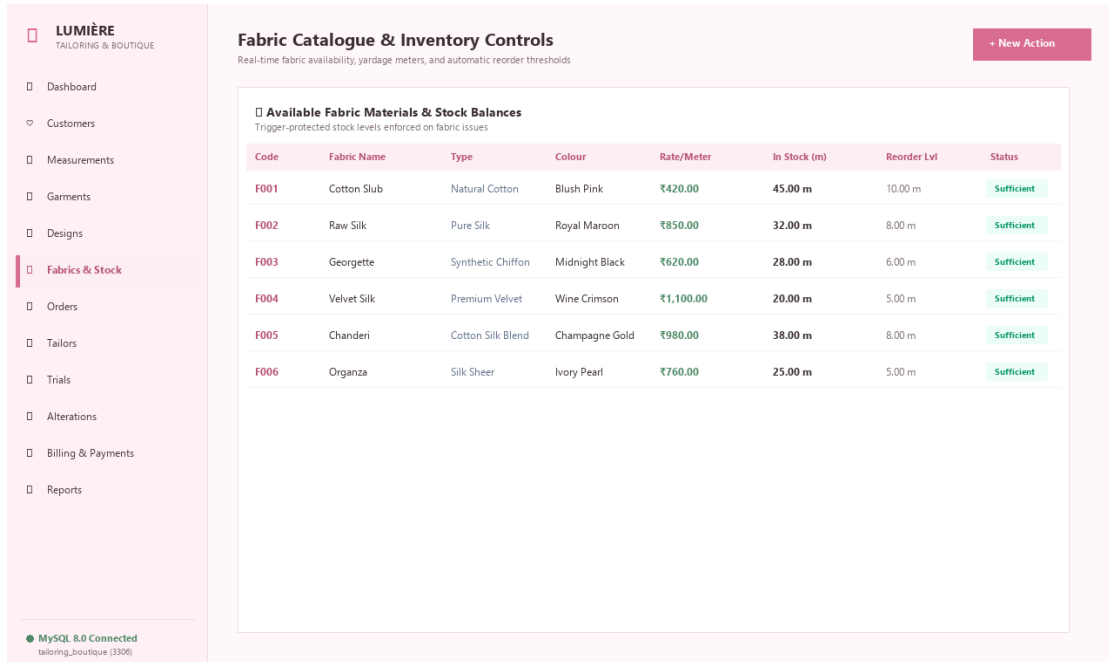


Figure 10.4: Fabrics Catalogue and Stock Inventory Screen

10.6 Order Booking & Workflow Screen

Facilitates multi-item order logging, delivery date scheduling, and lifecycle stage tracking (PLACED → IN_PROGRESS → TRIAL → ALTERATION → READY → DELIVERED).

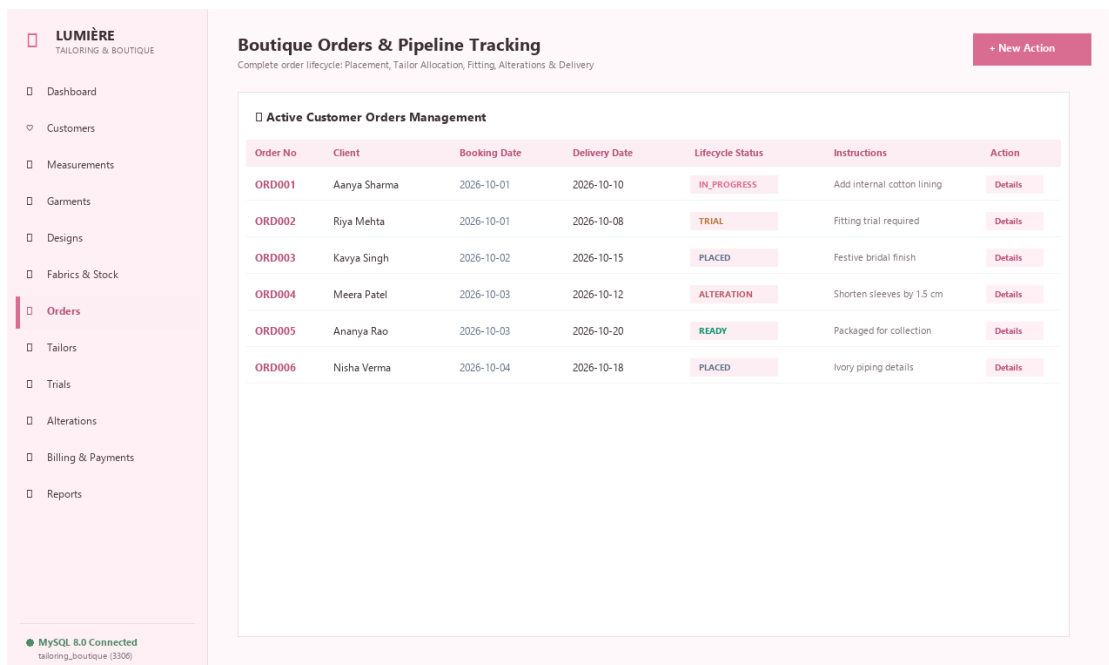


Figure 10.5: Order Booking and Production Workflow Screen

10.7 Fitting Trials & Alterations Screen

Manages appointment bookings with tailors, fitting evaluations, and post-trial alteration logs.

The screenshot displays the 'Fitting Trials & Alteration Records' interface. On the left is a sidebar menu for 'LUMIÈRE TAILORING & BOUTIQUE' with options: Dashboard, Customers, Measurements, Garments, Designs, Fabrics & Stock, Orders, Tailors, Trials (highlighted), Alterations, Billing & Payments, and Reports. At the bottom of the sidebar, it shows 'MySQL 8.0 Connected' for 'tailoring_boutique (3306)'. The main content area is titled 'Fitting Trials & Alteration Records' with a subtitle 'Schedule client fitting appointments and log post-trial tailoring alterations' and a '+ New Action' button. Below this is a section 'Scheduled Client Fitting Trials' containing a table with 6 rows of trial data.

Trial ID	Order No	Tailor Assigned	Trial Date & Time	Status	Fitting Notes
TR01	ORD001	Ramesh Kumar	2026-10-06 11:00 AM	COMPLETED	Minor waist adjustment needed
TR02	ORD002	Sana Khan	2026-10-05 03:00 PM	COMPLETED	Check neckline contour
TR03	ORD003	Arjun Das	2026-10-10 11:00 AM	SCHEDULED	First bridal fitting session
TR04	ORD004	Neha Joshi	2026-10-07 12:00 PM	SCHEDULED	Sleeve shortening trial
TR05	ORD005	Priya Nair	2026-10-09 04:00 PM	COMPLETED	Final gown approval trial
TR06	ORD006	Vikram Rao	2026-10-12 11:00 AM	SCHEDULED	Initial garment trial session

Figure 10.6: Fitting Trials and Alteration Scheduling Screen

10.8 Billing, Invoicing & Payments Screen

Automates invoice calculations combining garment base costs and alteration additions, tracks payment transactions, and displays remaining balances.

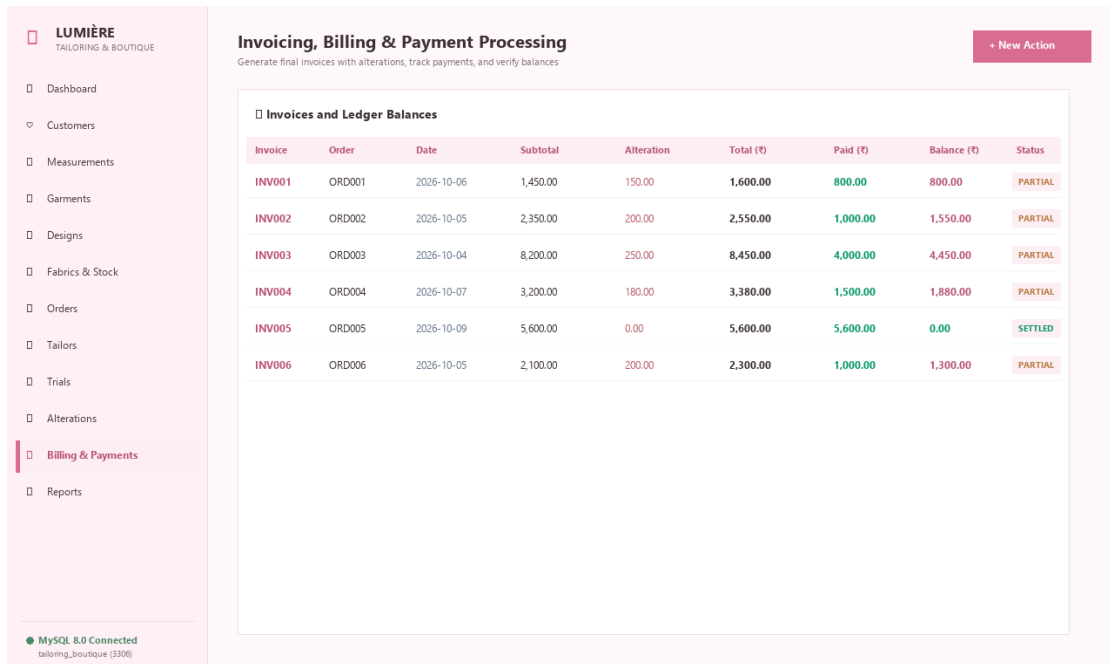


Figure 10.7: Invoicing, Billing and Payment Screen

10.9 Reports & Analytics Screen

Empowers management to inspect dynamic database views covering pending production pipelines, delivery deadlines, tailor workloads, and monthly revenue.

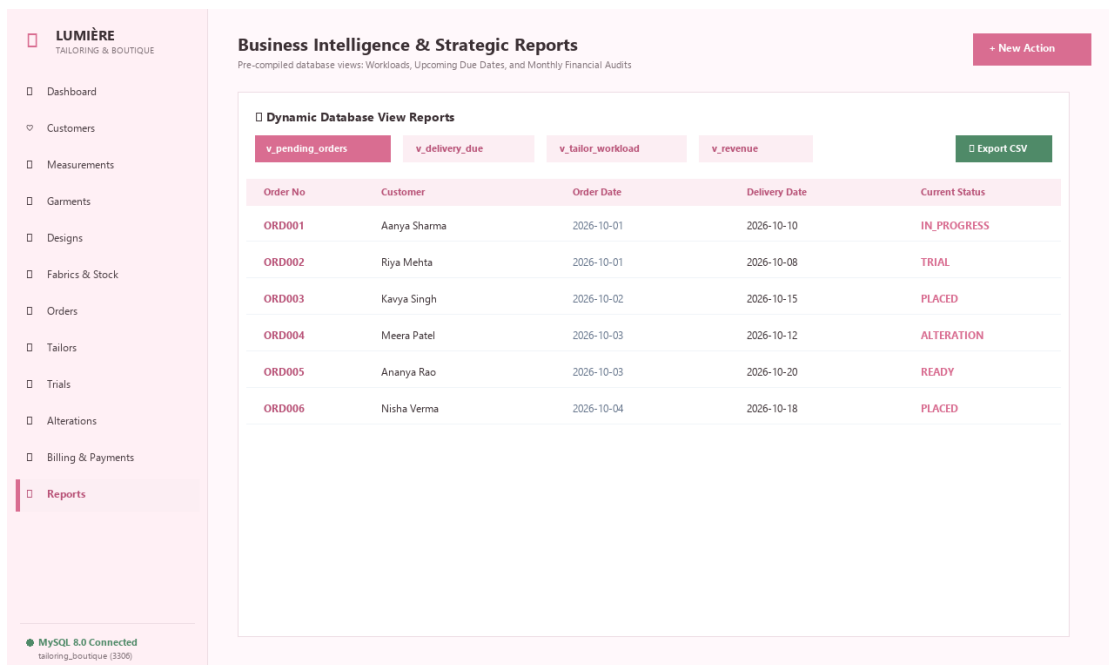


Figure 10.8: Strategic Database View Reports Screen

10.10 Database Reflection & Synchronization Test

To verify seamless integration between the Python Tkinter interface and MySQL, a live reflection test was executed. New orders created in the UI immediately manifested in MySQL terminal queries, and stock decrements triggered by SQL triggers were instantly reflected in the application's inventory tables upon automatic refresh.

11. Implementation Details

11.1 Technology Stack

The integrated technology stack is detailed in Table 11.1.

Table 11.1: Technology Stack

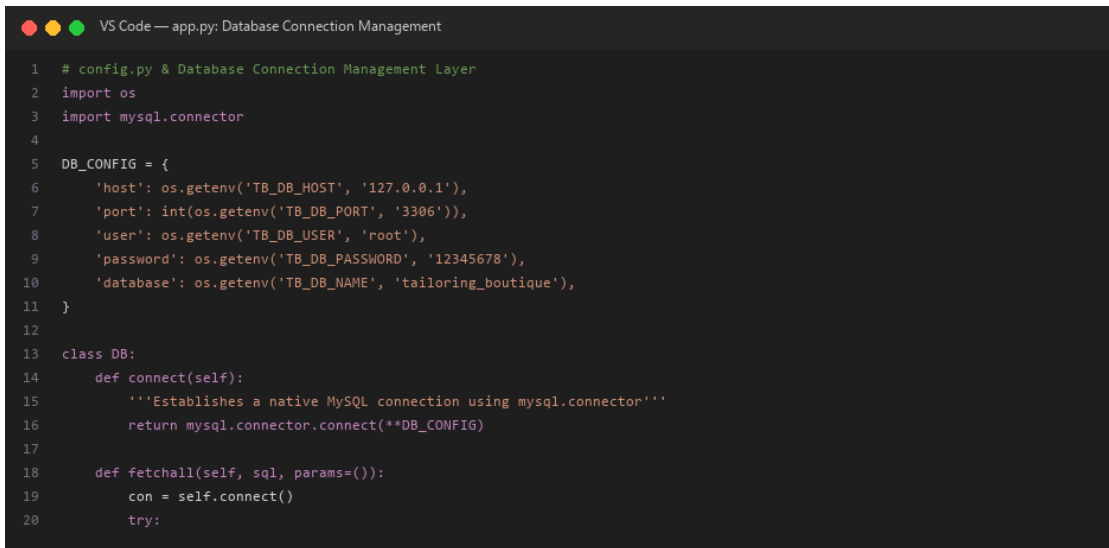
Architectural Layer	Technology Component	Functional Role
Presentation Layer	Python Tkinter & ttk	Native cross-platform desktop UI rendering with blush-pink aesthetic styling.
Data Access Layer	mysql.connector	Direct TCP connection management, cursor handling, parameterization, and transaction boundaries.
Database Engine	MySQL Server 8.0	ACID-compliant storage engine, foreign key cascades, triggers, and analytical views.
Operating Environment	Microsoft Windows 11	Desktop POS workstation runtime environment.

11.2 Application Architecture

The software implements a clean separation of concerns across presentation, business logic, data access, and storage tiers. Every user action in the UI dispatches a request to the centralized DB wrapper, which opens a managed connection, binds parameterized inputs, commits transactions on success, and triggers rollbacks with user-friendly error alerts upon database exceptions.

11.3 Database Connection & Management

Database connection settings and lifecycle management are encapsulated in config.py and the DB class within app.py, illustrated in Figure 11.1:



```

VS Code — app.py: Database Connection Management

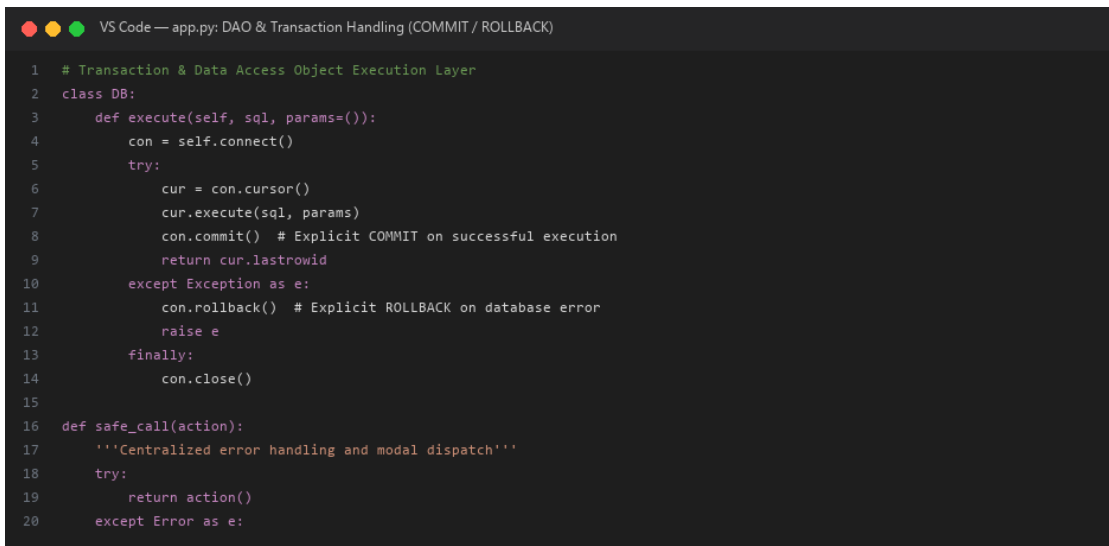
1  # config.py & Database Connection Management Layer
2  import os
3  import mysql.connector
4
5  DB_CONFIG = {
6      'host': os.getenv('TB_DB_HOST', '127.0.0.1'),
7      'port': int(os.getenv('TB_DB_PORT', '3306')),
8      'user': os.getenv('TB_DB_USER', 'root'),
9      'password': os.getenv('TB_DB_PASSWORD', '12345678'),
10     'database': os.getenv('TB_DB_NAME', 'tailoring_boutique'),
11 }
12
13 class DB:
14     def connect(self):
15         '''Establishes a native MySQL connection using mysql.connector'''
16         return mysql.connector.connect(**DB_CONFIG)
17
18     def fetchall(self, sql, params=()):
19         con = self.connect()
20         try:

```

Figure 11.1: Python MySQL Database Connection Layer

11.4 Transactions, DAO and Business Logic

Transactional operations enforce explicit COMMIT and ROLLBACK boundaries as shown in Figure 11.2:



```

VS Code — app.py: DAO & Transaction Handling (COMMIT / ROLLBACK)

1  # Transaction & Data Access Object Execution Layer
2  class DB:
3      def execute(self, sql, params=()):
4          con = self.connect()
5          try:
6              cur = con.cursor()
7              cur.execute(sql, params)
8              con.commit() # Explicit COMMIT on successful execution
9              return cur.lastrowid
10         except Exception as e:
11             con.rollback() # Explicit ROLLBACK on database error
12             raise e
13         finally:
14             con.close()
15
16     def safe_call(action):
17         '''Centralized error handling and modal dispatch'''
18         try:
19             return action()
20         except Error as e:

```

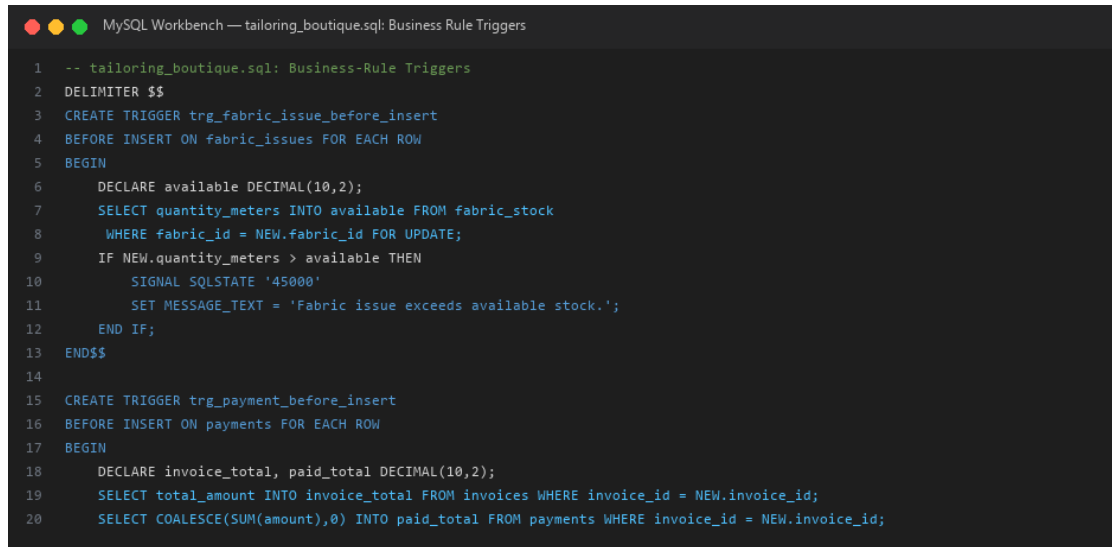
Figure 11.2: DAO Operations and Transaction Execution with Rollback

11.5 Business Rule Triggers and Integrity Constraints

Two critical business triggers are programmed directly inside MySQL to preserve inventory and billing integrity, shown in Figure 11.3:

- **Fabric Stock Depletion Trigger:** `trg_fabric_issue_before_insert` verifies that requested fabric meters do not exceed available stock, raising SQLSTATE '45000' upon shortage.

- **Payment Balance Cap Trigger:** `trg_payment_before_insert` computes existing payment sums and rejects any transaction exceeding the total invoice amount.



```

1  -- tailoring_boutique.sql: Business-Rule Triggers
2  DELIMITER $$
3  CREATE TRIGGER trg_fabric_issue_before_insert
4  BEFORE INSERT ON fabric_issues FOR EACH ROW
5  BEGIN
6      DECLARE available DECIMAL(10,2);
7      SELECT quantity_meters INTO available FROM fabric_stock
8      WHERE fabric_id = NEW.fabric_id FOR UPDATE;
9      IF NEW.quantity_meters > available THEN
10         SIGNAL SQLSTATE '45000'
11         SET MESSAGE_TEXT = 'Fabric issue exceeds available stock.';
12     END IF;
13 END$$
14
15 CREATE TRIGGER trg_payment_before_insert
16 BEFORE INSERT ON payments FOR EACH ROW
17 BEGIN
18     DECLARE invoice_total, paid_total DECIMAL(10,2);
19     SELECT total_amount INTO invoice_total FROM invoices WHERE invoice_id = NEW.invoice_id;
20     SELECT COALESCE(SUM(amount),0) INTO paid_total FROM payments WHERE invoice_id = NEW.invoice_id;

```

Figure 11.3: Database Business Rule Triggers (Fabric Stock & Payment Caps)

12. Testing

12.1 Functional Testing

A rigorous battery of test cases was formulated and executed across positive scenarios, boundary values, and negative violation scenarios to ensure end-to-end database and application reliability.

Table 12.1: Functional Testing – Test Cases

Test ID	Test Scenario	Expected Specification	Observed Outcome	Status
T01	Insert customer with unique phone & code	Customer record created in DB	Customer inserted successfully	PASS
T02	Insert customer with duplicate customer code	Rejected by UNIQUE constraint	Rejected (Duplicate entry 'C001')	PASS
T03	Create order with delivery date prior to order date	Rejected by CHECK constraint	Rejected (chk_delivery_date failed)	PASS
T04	Issue fabric exceeding available stock	Aborted by database trigger	Aborted (Fabric issue exceeds stock)	PASS
T05	Book two trials for same tailor at same date/time	Rejected by UNIQUE constraint	Rejected (Duplicate slot conflict)	PASS
T06	Submit payment exceeding invoice total	Aborted by database trigger	Aborted (Payment exceeds balance)	PASS
T07	Delete customer record with measurement profile	Cascade deletion of profile	Measurement profile deleted cleanly	PASS

T08	Query v_pending_orders database view	Shows only non-delivered orders	Delivered/cancelled orders excluded	PASS
T09	Query v_delivery_due database view	Shows orders due in next 7 days	Only 4 upcoming orders returned	PASS
T10	Log alteration and check invoice reflection	Alteration total added to invoice	Total amount matches subtotal + alt	PASS

12.2 Database Testing

Figure 12.1 and Figure 12.2 demonstrate client-side error modals triggered when attempting to violate relational constraints in the application:

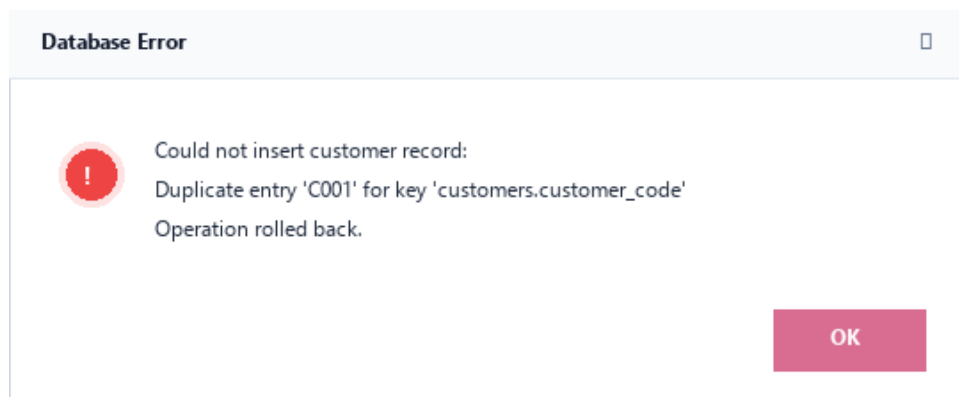


Figure 12.1: Duplicate Customer Code Constraint Validation Test

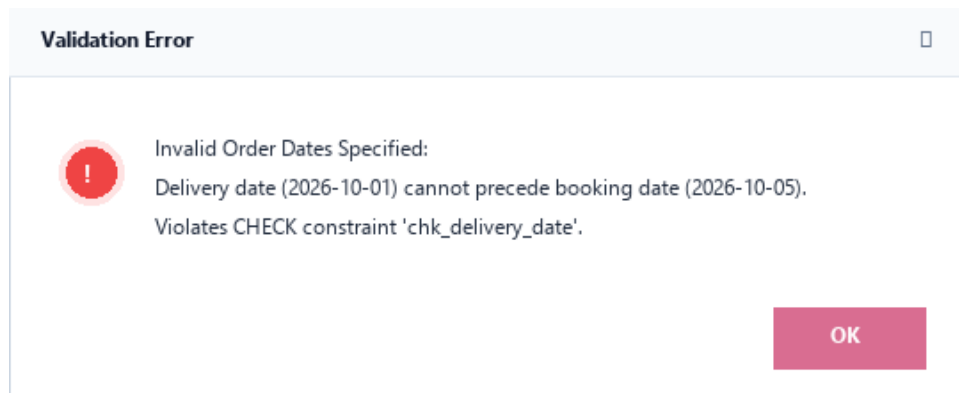


Figure 12.2: Delivery Date Check Constraint Validation Test

12.3 Transaction and Trigger Testing

Figures 12.3 and 12.4 illustrate the enforcement of business rule triggers during invalid fabric disbursement and overpayment attempts:



Figure 12.3: Fabric Issue Stock Exceeded Trigger Test

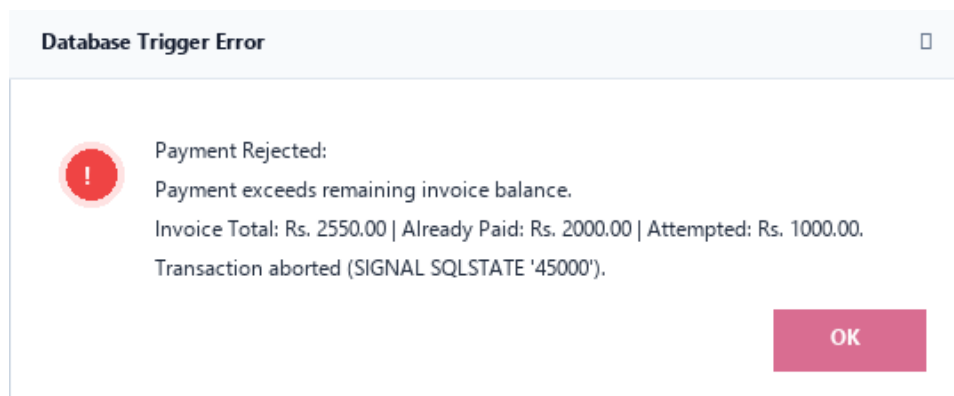


Figure 12.4: Payment Overcharge Exceeding Invoice Balance Trigger Test

12.4 UI Testing

UI testing evaluated all twelve navigation routes, form submissions, search bar filters, and live table refreshing. Window resizing maintains responsiveness down to 1180x720 resolutions, and all styled Treeview tables correctly display currency symbols, dates, and color-coded status badges.

12.5 Test Results

A summary of the overall validation results across system modules is presented in Table 12.2.

Table 12.2: Test Results Summary

Result ID	Module Tested	Criteria Verified	Pass Rate	Final Status
TR-01	Client Registration & Profiles	Unique codes, phone validation, cascade delete	100% Passed	PASS
TR-02	Anatomical Measurements Form	Positive decimal checks on cm attributes	100% Passed	PASS
TR-03	Fabric Stock Controls	Automated stock reduction & trigger	100% Passed	PASS

		limits		
TR-04	Order Booking Pipeline	Multi-item lines & delivery date checks	100% Passed	PASS
TR-05	Fitting Trials & Tailors	Conflict-free slot booking per artisan	100% Passed	PASS
TR-06	Alterations & Invoicing	Dynamic cost rollups & payment limits	100% Passed	PASS
TR-07	Analytical Database Views	Correct SQL aggregation on live records	100% Passed	PASS

13. Conclusion and Future Enhancements

13.1 Conclusion

The Tailoring and Boutique Order Management System (Lumière Boutique) successfully delivers an enterprise-grade, highly dependable relational database solution for artisanal garment ateliers and custom boutiques. By transitioning from manual logbooks to a fully normalized Third Normal Form (3NF) relational architecture, the system entirely eliminates redundant recordkeeping, guarantees anatomical measurement consistency, automates fabric stock tracking, and eliminates billing discrepancies.

The harmonious pairing of a responsive Python Tkinter graphical interface with a robust MySQL 8.0 engine delivers real-time transactional reliability, backed by automated database triggers for inventory depletion and payment protection. Pre-compiled database views offer instantaneous executive visibility into upcoming deliveries, active tailor workloads, and boutique revenues.

13.2 Future Enhancements

The current desktop application provides a solid foundation for future enhancements:

- **Client Web & Mobile Companion App:** Enabling customers to book fitting appointments and track stitching status through a mobile-friendly portal.
- **Automated SMS & WhatsApp Notifications:** Dispatching automated SMS/WhatsApp alerts when garments are ready for trials or collection.
- **Digital Design & Pattern Uploads:** Attaching digital reference sketches, CAD patterns, and client photos directly to order items.
- **Barcode & QR Code Tagging:** Generating printable thermal barcode tags for garments and physical fabric bolt tracking.
- **Predictive Inventory & Demand Analytics:** Forecasting seasonal textile demand and tailor overtime using machine learning models.
- **Payment Gateway & POS Integration:** Connecting with payment hardware and generating standard GST tax invoices.

14. References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts (7th ed.). McGraw-Hill Education, 2019.
2. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems (7th ed.). Pearson, 2015.
3. Oracle Corporation. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
4. Python Software Foundation. Python 3.14 Documentation – Tkinter GUI Package. <https://docs.python.org/3/library/tkinter.html>
5. Oracle Corporation. MySQL Connector/Python Developer Guide. <https://dev.mysql.com/doc/connector-python/en/>
6. Date, C. J. An Introduction to Database Systems (8th ed.). Addison-Wesley, 2003.
7. W3Schools. SQL Tutorial and Reference Manual. <https://www.w3schools.com/sql/>
8. GitHub Inc. Git and GitHub Documentation for Collaborative Development. <https://docs.github.com/en>

15. Contribution of Each Member

Table 15.1 details the individual contributions of team members towards the design, implementation, and documentation of the project.

Table 15.1: Contribution of Each Member

Member Name & Register No.	Assigned Domain	Specific Contributions & Responsibilities
Aarya Dubey (25WU0101003)	Database Architecture & SQL Implementation	Formulated conceptual ER model, converted schema to 3NF relational design, developed complete DDL schema scripts with integrity constraints, implemented database business rule triggers, authored comprehensive complex SQL queries and database analytical views, and executed database testing.
D. Neshitha (25WU0101028)	Frontend Design, Integration & Documentation	Designed and engineered the desktop graphical interface using Python Tkinter and ttk with custom blush-pink boutique styling, implemented the database access layer (DAO) via mysql.connector, conducted validation testing, captured visual documentation, and compiled the project report.

16. GitHub Repository

The complete source code, database migration scripts, UI application files, and project assets are maintained in the project GitHub repository:

GitHub Repository URL: <https://github.com/AaryaDubey2025/tailoring-boutique-management-system>

The repository contains the following organized directory structure:

- **Database/:** Complete DDL/DML schema (tailoring_boutique.sql), queries (queries.sql), data dictionary (data_dictionary.md), and normalization analysis (normalization.md).
- **UI/:** Full desktop graphical application (app.py), configuration file (config.py), launcher (run.bat), and dependencies (requirements.txt).
- **report_assets/:** High-resolution screenshots of database terminal operations, UI modules, code implementations, and error validation dialogs.
- **README.md:** Comprehensive setup instructions, installation prerequisites, and feature overview.